Deploying Machine Learning Models

From a trained notebook model to a reliable, observable, secure production service

Patterns · MLOps · Monitoring · LLMs

BUILD
Package, serve, containerise

SHIP
Automate, release, scale

RUN
Monitor, retrain, govern



Training a good model is only the beginning. Value appears when predictions reach real users, reliably, every day.

The gap

Many teams build models that never leave a notebook because the path to production is unclear.

The goal

Give you a complete, practical map of every decision between a trained model and a trusted service.

The promise

No heavy mathematics. Plain language, patterns, checklists and worked examples you can reuse.

GETTING STARTED

1 Data scientists

Who train strong models and now need to ship them without depending on a separate team.

2 ML engineers

Who own serving, pipelines and infrastructure and want a structured reference.

3 Software engineers

Who are integrating models into products and need to understand their quirks.

4 Platform and DevOps

Who run the clusters, registries and pipelines that models depend on.

5 Technical leads

Who must choose architectures, tools and team responsibilities with confidence.

6 Students and career changers

Who want an end to end view that bridges coursework and industry practice.

GETTING STARTED

What you should already know

- Basic Python and working with packages
- How a model is trained, validated and saved
- Common metrics such as accuracy, precision and recall
- Using a terminal and running scripts
- The idea of an API that receives and returns data
- Version control with Git at a basic level
- Helpful but optional: Docker and the cloud
- Helpful but optional: SQL and data pipelines

GETTING STARTED

Learning outcomes

1 Choose a pattern

Pick batch, online, streaming or edge deployment for a given business problem and defend the choice.

2 Package and serve

Turn a trained model into a versioned artifact behind a tested, documented API.

3 Containerise and scale

Build lean container images and run them on Kubernetes or managed cloud services.

4 Automate delivery

Design CI and CD pipelines with quality gates, registries and safe release strategies.

5 Observe and improve

Monitor systems, data and predictions, detect drift and trigger retraining.

6 Govern responsibly

Apply security, privacy, fairness and audit practices that stand up to scrutiny.

GETTING STARTED

Roadmap of the twenty sections

No.	Section	No.	Section
1	Foundations of deployment	11	Release strategies
2	Deployment patterns	12	Performance optimisation
3	Preparing the model	13	Monitoring and observability
4	Serving through APIs	14	Retraining and the model lifecycle
5	Containerisation	15	Edge and mobile deployment
6	Serving frameworks	16	Deploying large language models
7	Kubernetes and orchestration	17	Security, privacy and governance
8	Cloud and managed platforms	18	Reliability and cost
9	CI and CD for machine learning	19	End to end case study
10	Testing models for production	20	Checklists, glossary and next steps



Each section opens with a divider, teaches through concept, comparison and practice slides, and closes with a recap and an exercise.

GETTING STARTED

How the slides are designed to be read

Concept slides	Introduce one idea with a lead statement and three supporting cards.
Comparison slides	Place two options side by side and end with a clear verdict.
Flow slides	Show a process as numbered boxes connected by arrows.
Practice slides	Code, checklists, pitfalls and knowledge checks you can act on.
Recap slides	Summarise the section, set an exercise and preview what comes next.

Reading guide

Blue boxes carry core ideas. Orange boxes warn about traps. Yellow sticky notes hold tips worth remembering. Dark boxes give verdicts and next steps.

SECTION 1

01

Foundations of deployment

What it really means to put a model into production, and why it is harder than it looks.

IN THIS SECTION

- What deployment means
- Training versus serving
- Why projects stall
- The ML system landscape
- The deployment lifecycle
- Roles and maturity levels

SECTION 1 · FOUNDATIONS

What is model deployment?

DEFINITION

Deployment is the process of making a trained model available so that other systems or people can use its predictions reliably, securely and repeatedly in real conditions.

EXAMPLE

A churn model trained in a notebook becomes a nightly job that scores every customer and writes risk scores into the CRM, where account managers act on them each morning.

REMEMBER

A model is not deployed until someone outside the data team depends on its output.

SECTION 1 · FOUNDATIONS

Training world versus serving world

TRAINING

Experiment focused

- Static historical data loaded in bulk
- Runs for minutes or hours, then stops
- Success means a better validation score
- Failures are cheap and private
- One person controls the environment

SERVING

Product focused

- Live, messy data arriving continuously
- Runs indefinitely and must stay available
- Success means value, speed and reliability
- Failures are visible and costly
- Many teams and systems depend on it

VERDICT Deployment is the bridge between these worlds. Most of the work is engineering the serving side so it behaves as safely as the training side was measured.

SECTION 1 · FOUNDATIONS

Why so many models never reach production

1 No clear owner

Nobody is accountable for the model after training, so handover stalls.

2 Notebook only code

Logic lives in cells with hidden state that cannot be rerun reliably.

3 Training serving skew

Features are computed differently in production than in training.

4 Missing infrastructure

There is no registry, pipeline or serving platform to deploy onto.

5 Unclear value

The business metric the model should move was never agreed.

6 Risk and compliance

Legal, privacy or fairness questions surface late and block release.

“

The model is a small box in the middle of a very large system.

☆ Industry experience, popularised by the paper on hidden technical debt in machine learning systems, shows that model code is a minor share of a production ML system.

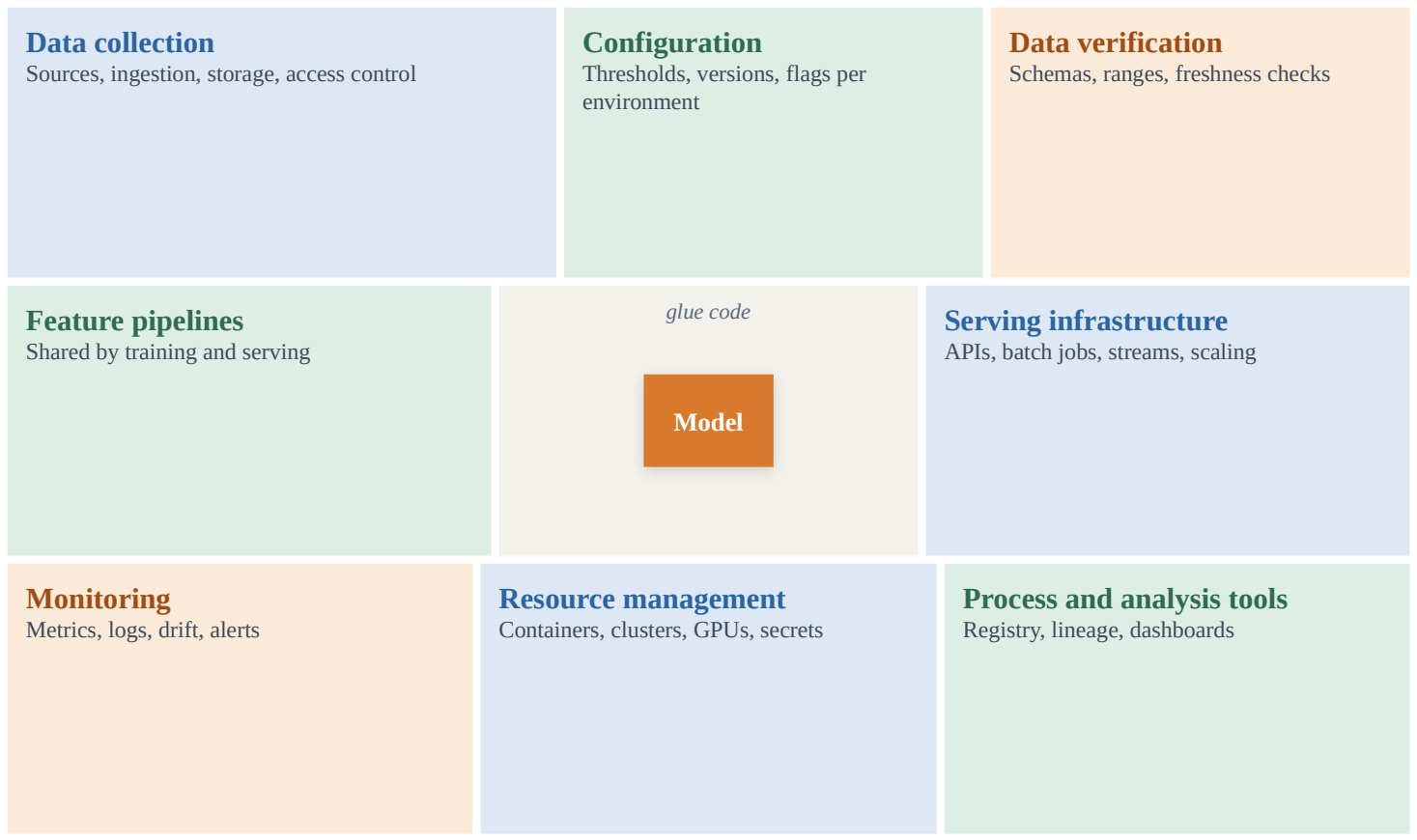
Structure of a production ML system

Data collection	Sources, ingestion, storage and access control for raw data.
Feature pipeline	Validation, transformation and feature storage shared by training and serving.
Model	The trained artifact plus its metadata, version and evaluation results.
Serving layer	APIs, batch jobs or streams that deliver predictions to consumers.
Infrastructure	Containers, clusters, networking, secrets and compute resources.
Monitoring	Metrics, logs, drift checks, alerts and feedback for retraining.

Where effort goes

Expect to spend far more time on data, configuration, serving and monitoring than on the model itself. Plan your team and schedule around that reality.

The model really is the small box in the middle



Tile sizes indicate typical engineering effort, after Sculley et al., Hidden Technical Debt in ML Systems (2015).

FAILURES THAT START OUTSIDE THE MODEL

Feature pipeline
An upstream team changed a field from pounds to pence. Predictions went wrong; the service stayed green.

Monitoring
Nobody watched input drift, so a broken sensor fed zeros for six weeks.

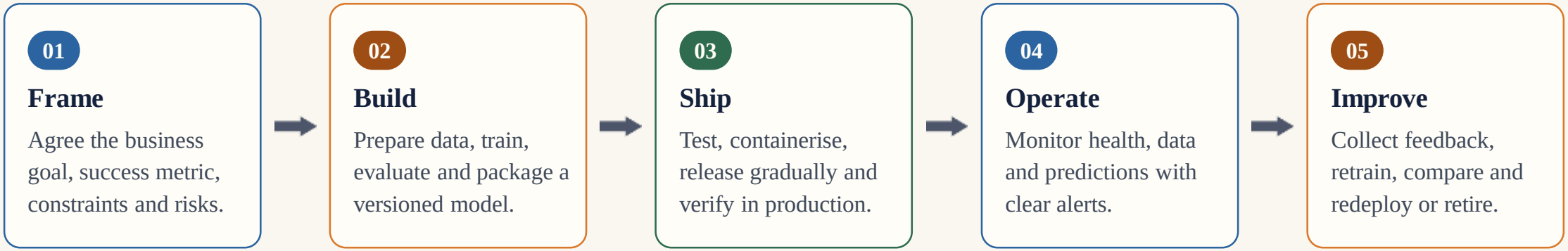
Configuration
A typo pointed production at last year's model version.

REMEMBER

Plan the boxes around the model before polishing the model.

SECTION 1 · FOUNDATIONS

The end to end deployment lifecycle



This is a loop, not a line. Every production model eventually returns to the Build stage as data and the world change.

SECTION 1 · FOUNDATIONS

Deployment is a product decision

Before writing serving code, decide who uses the prediction, how fast they need it, and what happens when it is wrong.

Consumer

A human dashboard, another service, a mobile app or a batch report each demand a different design.

Latency need

Milliseconds for a checkout fraud check, hours for a weekly demand forecast.

Cost of error

A wrong film recommendation is harmless. A wrong medical triage score is not.

Who does what in a deployment

Role	Typical responsibilities
Data scientist	Problem framing, features, training, evaluation, model documentation
ML engineer	Packaging, serving, pipelines, performance, model registry
Data engineer	Ingestion, data quality, feature pipelines, storage
Platform or DevOps	Clusters, CI and CD tooling, networking, secrets, cost controls
Product owner	Business metric, acceptance criteria, release decisions
Risk and compliance	Privacy, fairness, regulatory and audit sign off



In small teams one person may hold several roles. What matters is that every responsibility has a named owner.

MLOps maturity levels

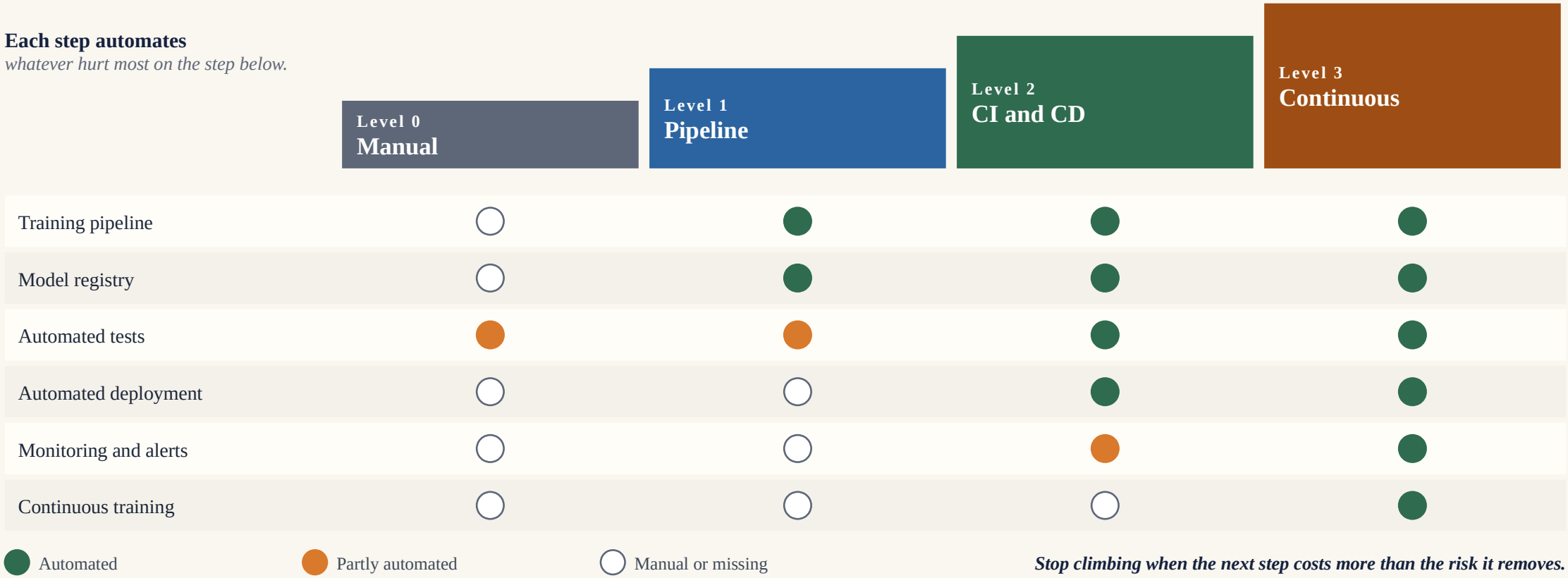
Level	What it looks like	Main risk
Level 0	Manual training, manual handover, scripts run by hand	Slow, fragile, unrepeatable releases
Level 1	Automated training pipeline and model registry	Deployment still manual and error prone
Level 2	CI and CD for code, data and models with tests	Monitoring may lag behind delivery
Level 3	Continuous training, monitoring and automatic gates	Complexity and cost of the platform



These levels echo the widely cited Google Cloud MLOps maturity description. Aim for the lowest level that safely meets your needs, then climb.

Climbing the MLOps maturity staircase, one capability at a time

Each step automates
whatever hurt most on the step below.



Stop climbing when the next step costs more than the risk it removes.

SECTION 1 · FOUNDATIONS

Foundational mistakes to avoid

PITFALL	BETTER PRACTICE
⚠ Treating deployment as a final step after the model is finished.	✓ Design for deployment from day one: inputs, latency, owners and monitoring.
⚠ Optimising only for offline accuracy.	✓ Track the business metric and operational metrics alongside model quality.
⚠ Building custom infrastructure for the first model.	✓ Start with managed or proven tools and add complexity only when needed.
⚠ Assuming the model will stay accurate forever.	✓ Plan monitoring and retraining before the first release.

Knowledge check

? A model scores 95 percent accuracy offline but is never used by the sales team. Has it been deployed?

MODEL ANSWER

No. Deployment means the predictions are available and relied upon in a real workflow. A high offline score with no consumer delivers no value. The team should identify the consumer, the delivery channel and the action the prediction triggers.

WHY IT MATTERS

It reframes success from model quality to delivered business value.

Section recap

KEY TAKEAWAYS

- Deployment makes predictions reliably usable by real consumers
- Serving conditions differ sharply from training conditions
- Most project failures are organisational and infrastructural
- The model is a small part of a larger system
- Deployment is a continuous lifecycle, not a single event

TRY THIS

Pick one model you have built. Write down its consumer, required latency, cost of a wrong prediction and who would own it in production.

NEXT UP

Section 2: the main deployment patterns and how to choose between them.

SECTION 2

02

Deployment patterns

The handful of shapes that almost every production model takes, and how to pick the right one.

IN THIS SECTION

- Batch prediction
- Online real time serving
- Streaming inference
- Edge and on device
- Embedded and serverless
- Choosing and combining

SECTION 2 · PATTERNS

Batch prediction

Batch prediction scores a large set of records on a schedule and stores the results for later use. It is the simplest and often the most robust pattern.

How it works

A scheduled job loads data, runs the model over every row and writes predictions to a table or file.

Great for

Churn scores, demand forecasts, lead scoring, monthly risk ratings and recommendations refreshed nightly.

Trade off

Predictions can be stale and you score records nobody may ever look at.

SECTION 2 · PATTERNS

Online real time serving

Online serving answers one request at a time through an API, returning a prediction in milliseconds while the user or system waits.

How it works

A client sends features in a request, the service runs the model and returns a response immediately.

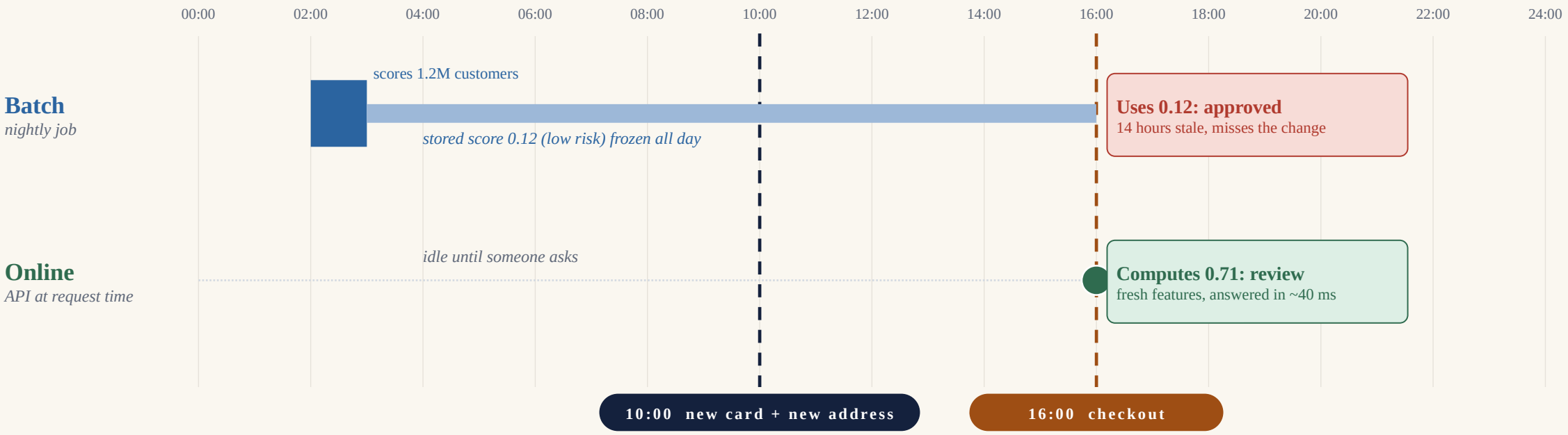
Great for

Fraud checks at payment, search ranking, dynamic pricing, chat assistants and personalised content.

Trade off

Needs high availability, tight latency budgets, scaling and careful feature freshness.

Batch versus online, told through one customer's day



BATCH PAYS OFF WHEN

Inputs change slower than the schedule, and a few hours of staleness costs nothing. Cheap, simple, easy to rerun.

ONLINE PAYS OFF WHEN

The decision depends on what just happened. You pay for uptime, a latency budget and fresh features.

SECTION 2 · PATTERNS

Streaming inference

Streaming inference consumes events from a message stream, scores them continuously and publishes results back to another stream.

How it works

Events arrive on a platform such as Kafka, a consumer applies the model and emits predictions downstream.

Great for

Sensor anomaly detection, clickstream personalisation, transaction monitoring and log analysis.

Trade off

Adds stream processing complexity, ordering concerns and stateful feature windows.

SECTION 2 · PATTERNS

Edge and on device deployment

Edge deployment runs the model on the device where data is created, such as a phone, camera, vehicle or factory gateway.

How it works

A compressed model ships inside an app or firmware and runs locally without a network round trip.

Great for

Offline use, privacy sensitive data, very low latency vision and speech, and bandwidth savings.

Trade off

Limited memory and compute, harder updates and many hardware variations to support.

SECTION 2 · PATTERNS

Embedded model versus model as a service

EMBEDDED

Model inside the application

- Loaded directly in the app process
- No network hop, lowest latency
- Model updates require app redeploys
- Couples app and model dependencies

SERVICE

Model behind its own API

- Runs as a separate service
- Independent scaling and versioning
- Adds network latency and a dependency
- Reusable by many applications

VERDICT Embed for small, stable models owned by one team. Use a separate service when models change often or serve several consumers.

SECTION 2 · PATTERNS

Serverless inference

Serverless platforms run your prediction code on demand and scale to zero when idle, so you pay only for requests actually served.

How it works

You upload a function or container and the provider starts instances automatically as requests arrive.

Great for

Spiky or low traffic workloads, prototypes, internal tools and lightweight models.

Trade off

Cold starts add delay, memory and runtime limits apply, and GPU support varies by provider.

SECTION 2 · PATTERNS

Pattern comparison at a glance

Pattern	Latency	Freshness	Complexity	Typical example
Batch	Hours	Scheduled	Low	Nightly churn scores
Online	Milliseconds	Per request	Medium	Card fraud check
Streaming	Seconds	Continuous	High	Sensor anomaly alerts
Edge	Milliseconds	On device	High	Phone camera filter
Serverless	Variable	Per request	Low to medium	Internal text classifier



Latency here means how long until a prediction is available to the consumer, not only model compute time.

SECTION 2 · PATTERNS

A decision path for choosing a pattern

01

Need it now?

If the consumer waits for the answer, start from online or edge.

02

Offline use?

If devices lack connectivity or data is private, prefer edge.

03

Events flow?

If data arrives as continuous events, consider streaming.

04

Known inputs?

If all inputs exist in advance, batch is usually cheapest.

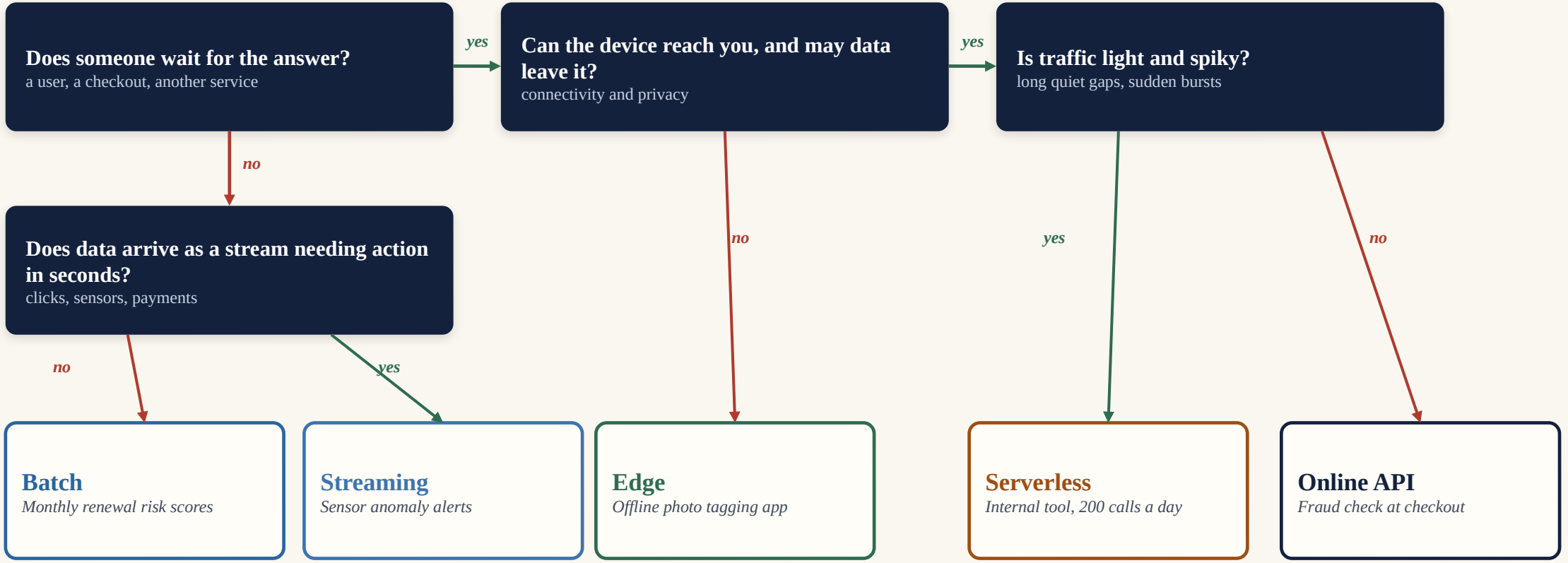
05

Traffic shape

Spiky and light traffic suits serverless.

When in doubt, begin with batch. It is easier to move from batch to online than to retreat from an overbuilt real time system.

The pattern decision path, drawn as a flowchart



! Each 'yes' along the top row adds operating cost. Answer for today's needs, not for the scale you hope to reach.

SECTION 2 · PATTERNS

Hybrid patterns are common



Mature systems often combine patterns: heavy work in batch, fresh adjustments online.

Precompute plus lookup

Batch jobs compute recommendations nightly and an API serves them from a fast key value store.

Two stage ranking

A cheap model retrieves candidates in batch and a richer model reranks them online.

Edge plus cloud

A small device model filters events and sends only uncertain cases to a larger cloud model.

SECTION 2 · PATTERNS

Pattern selection pitfalls

PITFALL	BETTER PRACTICE
⚠ Building real time serving because it sounds modern.	✓ Match the pattern to how quickly the business acts on predictions.
⚠ Ignoring how features will be available at request time.	✓ Confirm every feature can be computed within the latency budget.
⚠ Choosing edge without an update strategy.	✓ Plan over the air model updates and version checks before launch.
⚠ Using serverless for a large GPU model with steady load.	✓ Use dedicated instances when traffic is constant and models are heavy.

Knowledge check

? An insurer wants risk scores for every policy before monthly renewal letters are printed. Which pattern fits best?

MODEL ANSWER

Batch prediction. All policies are known in advance, the decision happens on a monthly schedule and nobody waits on a live response. A scheduled job can score every policy and write results to the renewals system at low cost.

WHY IT MATTERS

It shows that the business process, not technology fashion, should drive the pattern.

SECTION 2 · PATTERNS

Section recap

KEY TAKEAWAYS

- Batch is simple, cheap and robust for scheduled decisions
- Online serving answers live requests within milliseconds
- Streaming handles continuous events with low delay
- Edge runs locally for privacy, offline use and speed
- Serverless suits spiky, light workloads
- Hybrid designs mix patterns for cost and freshness

TRY THIS

List three products you use daily. For each, guess which deployment pattern powers its main prediction and explain why.

NEXT UP

Section 3: preparing the model artifact so it is fit for production.

SECTION 3

03

Preparing the model

Turning a trained model into a reproducible, portable and documented production artifact.

IN THIS SECTION

- From notebook to package
- Serialisation formats
- Bundling preprocessing
- Dependencies and reproducibility
- Validation and model cards
- Artifact checklist

SECTION 3 · PREPARING THE MODEL

From notebook to production package

01 Refactor

Move logic out of cells into functions and modules with clear inputs and outputs.

02 Pipeline

Wrap preprocessing and model into one object that is trained and saved together.

03 Serialise

Save the artifact in a stable format with a unique version identifier.

04 Document

Record data, metrics, limits and intended use in a model card.

05 Register

Store artifact and metadata in a model registry for controlled release.

A useful test: could a colleague rebuild and load your model from the repository alone, without asking you a single question?

Common model serialisation formats

Format	Typical use	Strength	Caution
Pickle	Python objects, sklearn	Simple, universal in Python	Unsafe to load untrusted files
Joblib	Large sklearn models	Efficient with big arrays	Same security risk as pickle
ONNX	Cross framework export	Portable, fast runtimes	Not every operator converts
SavedModel	TensorFlow and Keras	Includes graph and signatures	TensorFlow ecosystem only
TorchScript or export	PyTorch production	Runs without Python source	Tracing can miss dynamic logic
Safetensors	Model weights, LLMs	Safe and fast to load	Stores tensors, not full pipelines
Native booster	XGBoost, LightGBM	Stable across versions	Framework specific



Never unpickle a file from an untrusted source. Loading a pickle can execute arbitrary code on your server.

SECTION 3 · PREPARING THE MODEL

ONNX as a portability layer

ONNX, the Open Neural Network Exchange, is an open format that lets a model trained in one framework run in another runtime or on different hardware.

Why use it

Decouple training framework from serving runtime and unlock optimised engines such as ONNX Runtime.

Where it shines

Neural networks, many classical models through converters, CPU inference and mixed hardware fleets.

Watch out

Validate that converted outputs match the original within a small tolerance on real samples.

Bundle preprocessing with the model

train_pipeline.py

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import GradientBoostingClassifier
import joblib

prep = ColumnTransformer([
    ("num", StandardScaler(), ["age", "tenure"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["plan"]),
])
model = Pipeline([("prep", prep), ("clf", GradientBoostingClassifier())])
model.fit(X_train, y_train)
joblib.dump(model, "churn_model_v3.joblib")
```

One object

Scaling and encoding travel with the model, so serving cannot apply them differently.

Unknown categories

Ignoring unseen plan values prevents crashes when a new product launches.

Versioned name

The file name carries a version, but the registry remains the source of truth.

SECTION 3 · PREPARING THE MODEL

Training serving skew

DEFINITION

Training serving skew is any difference between how data is prepared during training and how it is prepared at prediction time, causing the model to see inputs it never learned from.

EXAMPLE

Training computed average spend over 30 days using complete data, but the live API computes it over whatever days are available, so new customers look very different in production.

REMEMBER

Share one feature implementation between training and serving whenever possible.

Training serving skew: one customer, two answers

Training pipeline

Offline SQL over complete history



30 day window: 25 empty days + 5 days with spend

£150 ÷ 30 days **£5 / day**

Same customer: joined 5 days ago, spent £150

Live API

Python code over the days available



only the 5 days that exist

£150 ÷ 5 days **£30 / day**

A 6× difference. The model learned that £30 a day means a loyal big spender. This brand-new customer is misjudged, and nothing errors.

One definition

Compute features with one shared library or a feature store, used by both training and serving.

Log what was served

Store the exact features each prediction used and compare their distribution with training data.

Test both paths

For a sample of customers, compute features both ways in CI and assert they match.

SECTION 3 · PREPARING THE MODEL

Reproducibility essentials

1 Pin dependencies

Lock exact library versions in a lock file so the same code always installs the same packages.

2 Version the data

Record the dataset snapshot or query and its hash used for each training run.

3 Fix random seeds

Set seeds for libraries that use randomness to make runs comparable.

4 Track experiments

Log parameters, metrics and artifacts with a tool such as MLflow.

5 Capture environment

Record Python version, operating system and hardware alongside the run.

6 Store the code commit

Tag every model with the Git commit that produced it.

Input validation and schemas

Every production model needs an explicit contract describing the fields it accepts, their types, ranges and allowed values.

Define the schema

Write it once with a library such as Pydantic, Pandera or Great Expectations and store it with the model.

Reject early

Return a clear error for missing fields, wrong types or impossible values instead of a silent bad prediction.

Log violations

Count and log rejected inputs. Rising rejections are often the first sign of an upstream change.

What belongs in a model card

Overview	Purpose, owner, version, date and intended users.
Data	Training sources, time range, preprocessing and known gaps.
Performance	Metrics overall and across important groups and segments.
Limitations	Situations where the model should not be trusted or used.
Ethics	Fairness checks, sensitive features and potential harms.
Operations	Inputs, outputs, latency, monitoring and retraining plan.



Why bother

Model cards, proposed by researchers at Google in 2019, make limitations visible to the people deploying and relying on a model. They also support audits and regulation.

Packaging pitfalls

PITFALL	BETTER PRACTICE
⚠ Saving only the estimator and rewriting preprocessing in the API.	✓ Save the full pipeline so transforms are identical everywhere.
⚠ Using loose requirements without versions.	✓ Pin every dependency and rebuild from a lock file.
⚠ Loading model files from shared drives at runtime.	✓ Pull versioned artifacts from a registry with checksums.
⚠ Leaving the model as an undocumented binary.	✓ Ship a model card and schema next to every artifact.

SECTION 3 · PREPARING THE MODEL

Production artifact checklist

- [ ] Preprocessing and model saved as one pipeline
- [ ] Stable serialisation format chosen deliberately
- [ ] Unique version and Git commit recorded
- [ ] Training data snapshot referenced
- [ ] Dependencies pinned in a lock file
- [ ] Input and output schema defined
- [ ] Evaluation metrics stored with the artifact
- [ ] Model card written and reviewed
- [ ] Load test: artifact loads in a clean environment
- [ ] Registered in the model registry

Section recap

KEY TAKEAWAYS

- Refactor notebook code into reusable modules
- Choose serialisation formats with security and portability in mind
- Bundle preprocessing to prevent training serving skew
- Pin dependencies and version data for reproducibility
- Validate inputs against an explicit schema
- Document every model with a model card

TRY THIS

Take an existing notebook model and save it as a single pipeline with a schema and a one page model card.

NEXT UP

Section 4: exposing the model through a reliable API.

SECTION 4

04

Serving through APIs

Designing prediction endpoints that are fast, predictable, documented and safe to depend on.

IN THIS SECTION

- REST and gRPC
- Request and response design
- A FastAPI service
- Health checks and versioning
- Concurrency and batching
- Errors and documentation

REST versus gRPC for model serving

REST

JSON over HTTP

- Human readable and easy to debug
- Works in every language and browser
- Larger payloads and slower parsing
- Ideal for public and general APIs

GRPC

Protocol Buffers over HTTP/2

- Compact binary messages
- Strong typed contracts from schema files
- Supports streaming in both directions
- Ideal for high volume internal traffic

VERDICT Start with REST for simplicity and reach. Move hot internal paths to gRPC when payload size or latency becomes a measured problem.

Designing the request and response

A good prediction API is boring: explicit fields, clear types, stable versions and useful metadata in every response.

Request

Named features with types, an optional request identifier and no hidden defaults.

Response

Prediction, probability or score, model version and the request identifier.

Metadata

Timestamps and latency help debugging, audits and joining predictions to outcomes later.

A boring, excellent prediction API, annotated line by line

```
POST /v1/predict

{
  "request_id": "a1f3-9c2e",                    (1)
  "features": {
    "tenure_months": 14,                        (2)
    "plan": "premium",
    "monthly_spend_gbp": 42.50
  }
}
```

→

```
200 OK

{
  "request_id": "a1f3-9c2e",                    (3)
  "prediction": "churn",
  "probability": 0.73,                          (4)
  "model_version": "churn-2.4.1",               (5)
  "latency_ms": 18,                             (6)
  "timestamp": "2026-11-01T18:20:05Z"           (6)
}
```

1. **Request ID.** Created by the caller so the prediction can be joined to the real outcome weeks later.
2. **Named, typed features.** Units in the names and no hidden defaults: a missing field is an error, not a zero.
3. **Echoed ID.** Lets the caller's logs and yours line up during incidents.
4. **Probability, not just a label.** Callers can apply their own threshold without a new model.
5. **Model version.** Answers 'which model said this?' for debugging, audits and rollback.
6. **Latency and timestamp.** Feed latency objectives and time-based joins to outcomes.

SECTION 4 · SERVING APIS

A minimal FastAPI prediction service

app.py

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import joblib

app = FastAPI(title="Churn API", version="3.0")
model = joblib.load("churn_model_v3.joblib")

class Customer(BaseModel):
    age: int = Field(ge=18, le=110)
    tenure: float = Field(ge=0)
    plan: str

@app.post("/v3/predict")
def predict(c: Customer):
    p = model.predict_proba([c.model_dump()])[0][1]
    return {"churn_probability": round(p, 4), "model_version": "3.0"}
```

Load once

The model loads at startup, not on every request, which saves seconds per call.

Validated input

Pydantic rejects ages outside a sensible range with a clear error message.

Versioned path

The version in the URL lets old clients keep working during upgrades.

Health, readiness and liveness

Orchestrators and load balancers need simple endpoints to know whether your service is alive and ready to receive traffic.

Liveness

Answers whether the process is running. If it fails repeatedly, the platform restarts the container.

Readiness

Answers whether the model is loaded and dependencies reachable. Traffic waits until it passes.

Startup

Gives large models extra time to load before liveness checks begin to count failures.

SECTION 4 · SERVING APIS

Versioning prediction APIs

1 Version in the path

Use paths such as v1 and v2 so clients choose explicitly when to upgrade.

2 Echo model version

Return the exact model version in every response for traceability.

3 Separate API and model

API contract versions change rarely. Model versions can change often behind them.

4 Deprecate politely

Announce end dates, log callers of old versions and give migration notes.

5 Never break silently

Changing a field meaning without a version bump is a breaking change.

6 Keep a changelog

Record what changed, why and which clients are affected.

SECTION 4 · SERVING APIS

Life of a prediction request

01 Receive
Gateway authenticates the caller and applies rate limits.

02 Validate
Schema checks reject malformed or impossible inputs.

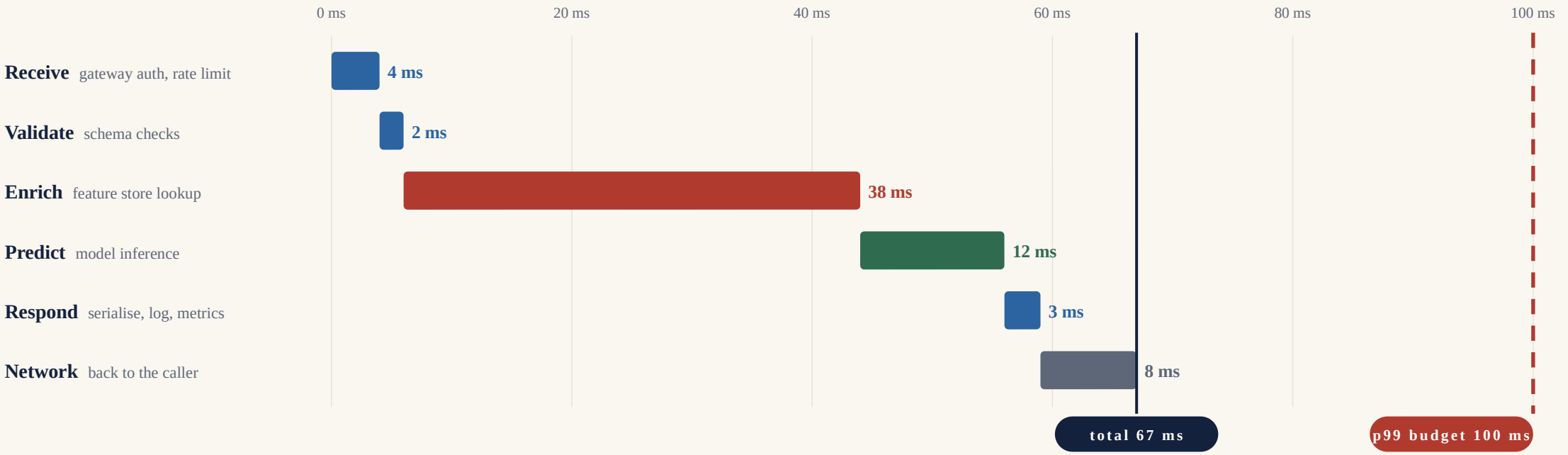
03 Enrich
Service fetches extra features from a feature store or cache.

04 Predict
Model runs, ideally within a strict time budget.

05 Respond
Return result, log inputs and outputs, emit metrics.

Measure the latency of each stage separately. Feature lookup is often slower than the model itself.

Where the milliseconds go inside one prediction request



57% of the time is the feature lookup

The model itself is only 18%.

WHAT IF FEATURES ARE CACHED?



Lookup 38 → 3 ms

Illustrative p95 timings

Concurrency, workers and async

Model inference is usually CPU or GPU bound, so concurrency settings matter more than clever code for throughput.

Workers

Run several worker processes so multiple requests use multiple CPU cores in parallel.

Async input and output

Use async handlers for network calls such as feature lookups, not for heavy model compute.

Thread pools

Offload blocking model calls to a thread or process pool so the event loop stays responsive.

SECTION 4 · SERVING APIS

Request batching

DEFINITION

Request batching groups several incoming requests into one model call, so hardware such as a GPU processes many inputs at once and throughput rises sharply.

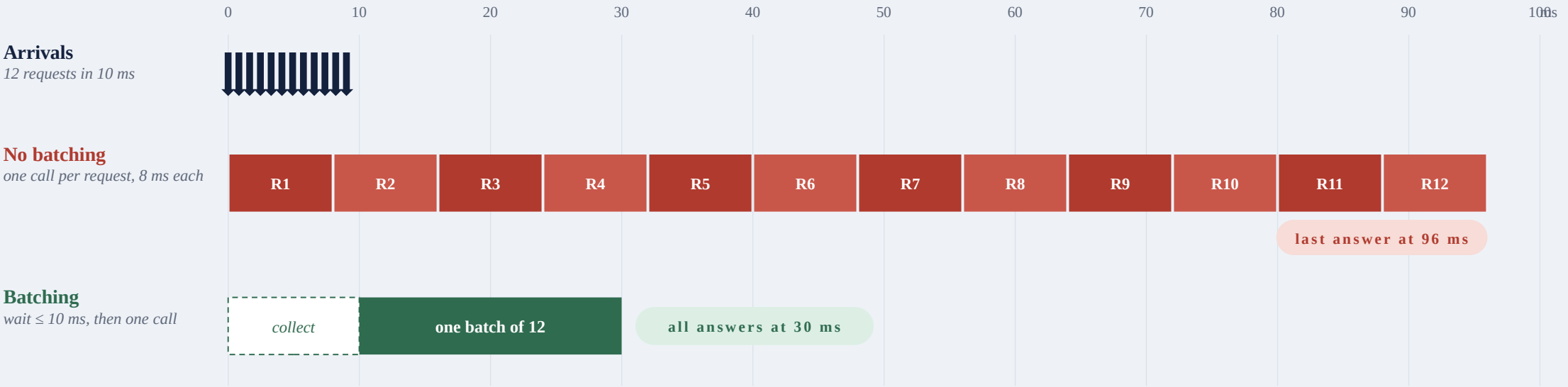
EXAMPLE

A server waits up to 10 milliseconds to collect as many as 32 image requests, runs them as one batch on the GPU, then returns each result to its caller.

REMEMBER

Batching trades a little latency per request for much higher total throughput.

Request batching on a timeline: twelve requests, two strategies



THROUGHPUT ON ONE GPU

Batch size	Time per call	Requests per second
1	8 ms	**125**
32	20 ms	**1,600**

Illustrative timings

TWO KNOBS TO TUNE

Max batch size · max wait

Raise them for throughput; cap the wait so p99 latency stays inside the budget.

Error handling that callers can act on

Status	Meaning	Model service example
400	Bad request	Field has the wrong type or is missing
401 or 403	Not authorised	Missing or invalid API key or token
422	Unprocessable input	Age of 250 fails the schema range check
429	Too many requests	Caller exceeded its rate limit
500	Server error	Unexpected exception inside the model code
503	Unavailable	Model still loading or overloaded
504	Timeout	Feature lookup exceeded its time budget



Return a short error code and message in the body. Never return stack traces or internal paths to callers.

SECTION 4 · SERVING APIS

API pitfalls

PITFALL	BETTER PRACTICE
⚠ Loading the model inside the request handler.	✓ Load once at startup and reuse it for every request.
⚠ Returning only a bare number.	✓ Return score, label, model version and request identifier.
⚠ No timeouts on feature lookups.	✓ Set strict timeouts and a fallback when dependencies are slow.
⚠ Undocumented endpoints.	✓ Publish OpenAPI documentation and example requests.

Section recap

KEY TAKEAWAYS

- Choose REST for reach and gRPC for heavy internal traffic
- Design explicit, versioned requests and responses
- Load models once and validate every input
- Expose liveness, readiness and startup checks
- Use workers and batching to raise throughput
- Return clear, safe error codes

TRY THIS

Wrap your packaged model in a FastAPI service with a schema, a health endpoint, a versioned path and three example requests.

NEXT UP

Section 5: packaging the service into a container.

SECTION 5

05

Containerisation

Packaging your model service so it runs the same way on a laptop, a test server and a production cluster.

IN THIS SECTION

- Why containers
- Images and containers
- Anatomy of a Dockerfile
- Multi stage and lean images
- GPU images
- Registries, tags and scanning

SECTION 5 · CONTAINERS

Why containers matter for ML

It works on my machine stops being an excuse when the machine itself is shipped.

Consistency

Code, libraries, system packages and the model run identically in every environment.

Isolation

Each service has its own dependencies, so library conflicts between models disappear.

Portability

The same image runs on Kubernetes, managed cloud services or a single virtual machine.

SECTION 5 · CONTAINERS

Images versus containers

IMAGE

The blueprint

- Read only template built from a Dockerfile
- Made of stacked, cached layers
- Stored and shared through a registry
- Identified by a tag and a digest

CONTAINER

The running instance

- Created from an image at runtime
- Has its own process, network and filesystem
- Disposable and replaceable at any time
- Many containers can share one image

VERDICT Build an image once, test it, then promote that exact image through every environment. Never rebuild for production.

Structure of a model service Dockerfile

1. **Base image.** Start from a small, official Python image with a fixed version tag.
2. **Working directory.** Set a clear folder inside the image where the application lives.
3. **Dependencies first.** Copy the lock file and install packages before copying code, to reuse cached layers.
4. **Application and model.** Copy source code and either the model file or a script that downloads it by version.
5. **Non root user.** Create and switch to an unprivileged user for safety.
6. **Start command.** Expose the port and launch the web server with a fixed number of workers.

Why 'dependencies first' makes rebuilds take seconds, not minutes

Dependencies first

after editing app.py

`CMD ["uvicorn", "src.app:app"]`	*rebuilt*
`USER app`	*rebuilt*
`COPY src/ ./src`	*rebuilt*
`RUN pip install -r requirements.txt`	*cached*
`COPY requirements.txt .`	*cached*
`WORKDIR /app`	*cached*
`FROM python:3.11-slim`	*cached*

Rebuild ≈ 10 seconds

Build order: bottom to top

Everything copied first

after editing app.py

`CMD ["uvicorn", "src.app:app"]`	*rebuilt*
`USER app`	*rebuilt*
`RUN pip install -r requirements.txt`	*rebuilt*
`COPY . .` ← code change breaks the cache here	*rebuilt*
`WORKDIR /app`	*cached*
`FROM python:3.11-slim`	*cached*

Rebuild ≈ 4 minutes

Rule: order layers from least to most often changed.

SECTION 5 · CONTAINERS

Multi stage builds

A multi stage build uses one stage with compilers and build tools, then copies only the finished results into a clean, small runtime stage.

Build stage

Installs compilers, builds wheels and prepares assets that need heavy tooling.

Runtime stage

Starts from a slim base and receives only the installed packages and app code.

Result

Smaller images download faster, start sooner and expose fewer components to attackers.

A multi-stage build ships only what runs

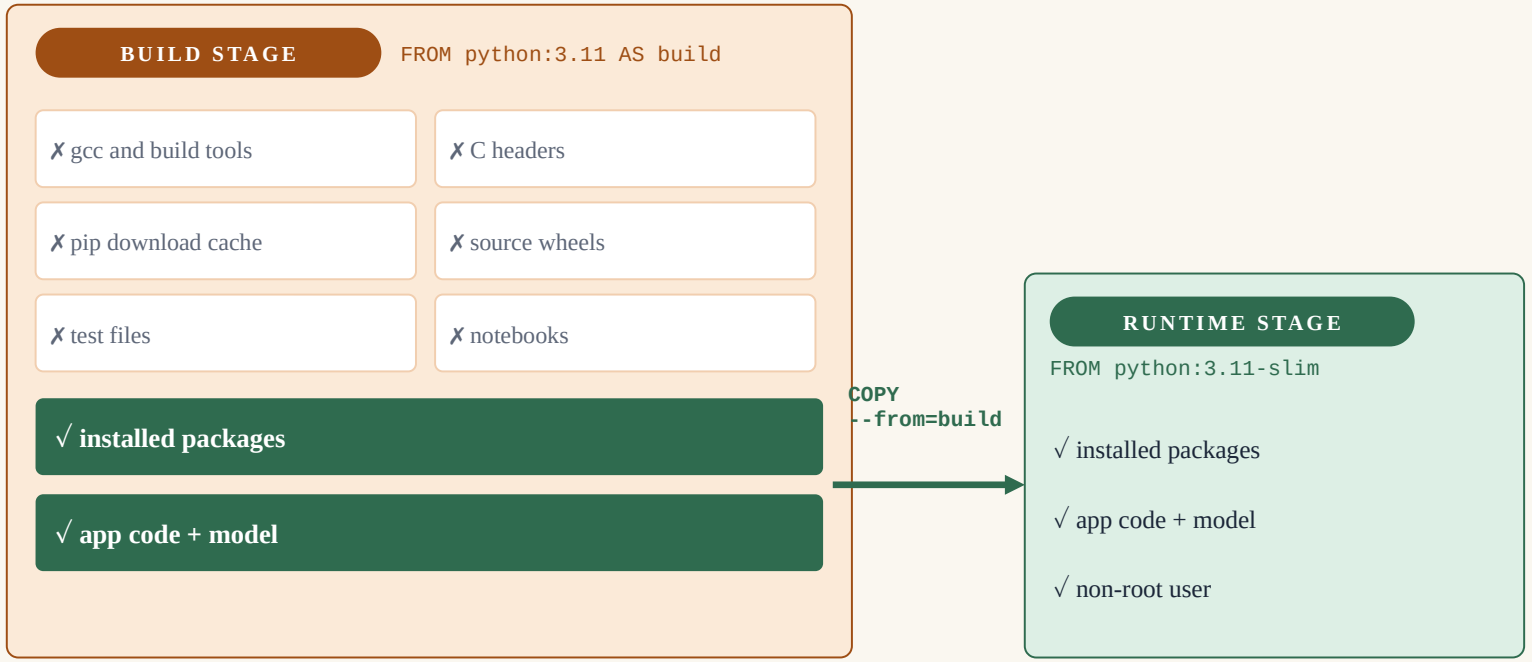


IMAGE SIZE AND PULL TIME

Single stage

2.1 GB · pull 21 s

Multi-stage

480 MB · pull 4.8 s

Pull time at 100 MB/s. Sizes illustrative.

```
FROM python:3.11 AS build
RUN pip install --prefix=/install -r requirements.txt
FROM python:3.11-slim    COPY --from=build /install /usr/local    COPY src/ /app/src
```

SECTION 5 · CONTAINERS

Keeping images lean

1 Slim bases

Prefer slim or distroless images over full operating system images.

2 Ignore files

Use a dockerignore file to exclude data, notebooks, caches and secrets.

3 No cache leftovers

Disable package manager caches during installation.

4 Combine steps

Group related install commands so temporary files do not persist in layers.

5 Only runtime libraries

Keep training and plotting libraries out of serving images.

6 Externalise big models

Pull large weights at startup from a registry or object storage when sensible.

GPU enabled images

GPU inference needs a compatible stack: the host driver, the container toolkit, and CUDA libraries inside the image that match your framework build.

Base choice

Start from vendor CUDA runtime images or framework images that already bundle matching libraries.

Version alignment

Framework, CUDA and driver versions must be compatible. Check support tables before upgrading.

Size awareness

GPU images are often several gigabytes, so cache them on nodes to avoid slow scaling.

Registries, tags and digests

Registry	Central store for images, such as ECR, Artifact Registry, ACR or Docker Hub.
Repository	One named collection of images for a service, for example churn api.
Tag	Human friendly label such as 3.0.1 or a Git commit identifier.
Digest	Content hash that uniquely and permanently identifies one exact image.

Golden rule

Deploy by digest or by an immutable tag. A tag like latest can point at different images over time, which makes incidents hard to reproduce.

Container security essentials

- Run as a non root user
- Use minimal, regularly updated base images
- Scan images for known vulnerabilities in CI
- Never bake secrets into image layers
- Inject secrets at runtime from a secret store
- Set read only filesystems where possible
- Limit CPU and memory for every container
- Sign images and verify signatures on deploy

SECTION 5 · CONTAINERS

Container pitfalls

PITFALL	BETTER PRACTICE
⚠ Copying the whole project folder, including datasets.	✓ Use a dockerignore file and copy only what serving needs.
⚠ Rebuilding images separately for each environment.	✓ Build once and promote the same digest everywhere.
⚠ Storing API keys in environment lines of the Dockerfile.	✓ Load secrets at runtime from a managed secret store.
⚠ Using the latest tag in production manifests.	✓ Pin immutable version tags or digests.

SECTION 5 · CONTAINERS

Section recap

KEY TAKEAWAYS

- Containers make model services consistent and portable
- Images are blueprints, containers are running instances
- Order Dockerfile steps to maximise cache reuse
- Multi stage builds produce small, safer images
- Align driver, CUDA and framework versions for GPUs
- Deploy immutable tags or digests and scan every image

TRY THIS

Containerise your FastAPI service, measure the image size, then apply three techniques from this section and measure again.

NEXT UP

Section 6: dedicated model serving frameworks.

SECTION 6

06

Serving frameworks

When a hand written API is not enough, specialised model servers bring batching, multi model hosting and speed.

IN THIS SECTION

- Why a model server
- Triton and framework servers
- KServe and Seldon
- BentoML and Ray Serve
- MLflow model serving
- A selection guide

Why use a dedicated model server?



Model servers solve the same problems every team eventually hits, so you do not have to reinvent them.

Built in batching

Dynamic batching groups requests automatically to use hardware efficiently.

Many models, one server

Load, unload and version several models with consistent endpoints.

Standard protocols

Common inference protocols make clients and tools interchangeable.

One model server, many models: from folder to endpoints

```
model_repository/
├─ fraud/
│  ├─ config.pbtxt
│  ├─ 1/model.onnx
│  └─ 2/model.onnx
├─ churn/
│  ├─ config.pbtxt
│  └─ 3/model.pt
└─ ranker/
   ├─ config.pbtxt
   └─ 1/model.plan
```

one folder per model, one number per version

MODEL SERVER

loads every model at start, watches for new versions

- Dynamic batching
- Several models share one GPU
- Version policy: serve latest 1 or N
- Health and metrics endpoints

! Add a model by adding a folder.

ENDPOINTS CLIENTS CALL

`POST /v2/models/fraud/infer`
latest fraud version

`POST /v2/models/fraud/versions/1/infer`
pin an older version

`POST /v2/models/churn/infer`
same shape, different model

`GET /v2/health/ready`
for the load balancer

`GET /metrics`
Prometheus scrape

Paths follow the open inference protocol (v2).

SECTION 6 · SERVING FRAMEWORKS

NVIDIA Triton Inference Server

Triton is an open source inference server that hosts models from many frameworks and is heavily optimised for GPUs as well as CPUs.

Multi framework

Serves TensorRT, ONNX, PyTorch, TensorFlow, Python and other backends side by side.

Performance

Dynamic batching, concurrent model execution and model ensembles out of the box.

Best for

High throughput GPU workloads, vision, speech and mixed model fleets.

Framework native servers

Some deep learning frameworks ship their own serving systems that understand their model formats natively.

TensorFlow Serving

Mature, high performance server for SavedModel files with versioned model directories.

TorchServe

PyTorch serving tool with handlers and management APIs. It moved to limited maintenance, so check its status first.

Trade off

Excellent for a single framework, less helpful when your team uses several.

SECTION 6 · SERVING FRAMEWORKS

KServe and Seldon Core on Kubernetes

These platforms add a model serving layer on top of Kubernetes, turning a short manifest into a scalable, observable prediction service.

KServe

Open source project offering an InferenceService resource, autoscaling and scale to zero, plus canary rollouts.

Seldon Core

Kubernetes serving with inference graphs, explainers and monitoring. Review its current licence before adopting.

Best for

Teams already running Kubernetes who want standardised serving for many models.

BentoML and Ray Serve

These Python first frameworks let you define services in code, then package and scale them with little infrastructure knowledge.

BentoML

Defines services in Python, builds deployable bundles and container images, with adaptive batching.

Ray Serve

Scales Python model code across a Ray cluster and composes multiple models into pipelines.

Best for

Python teams, complex multi model logic and fast iteration from prototype to production.

SECTION 6 · SERVING FRAMEWORKS

MLflow model serving

MLflow packages models in a standard format with a signature, and can serve them locally or build container images directly from the registry.

Model flavours

One format wraps sklearn, XGBoost, PyTorch and custom Python models consistently.

Registry link

Serve exactly the version promoted in the registry, keeping lineage intact.

Best for

Teams already tracking experiments in MLflow who want a short path to serving.

Serving framework comparison

Framework	Main strength	Best fit	Consider
Triton	GPU speed, many backends	High throughput inference	Configuration learning curve
TF Serving	Mature TensorFlow serving	TensorFlow only teams	Single framework
KServe	Kubernetes native, autoscaling	Platform teams on Kubernetes	Needs cluster expertise
Seldon Core	Inference graphs, explainers	Complex Kubernetes setups	Licence terms
BentoML	Python first packaging	Fast moving product teams	Less control of low level tuning
Ray Serve	Distributed composition	Multi model pipelines	Operating a Ray cluster
MLflow	Registry integrated	MLflow centred workflows	Fewer advanced serving features

Tool landscapes change quickly. Confirm project activity, licence and support before committing to any framework.

SECTION 6 · SERVING FRAMEWORKS

Choosing a serving framework

01 Workload

GPU heavy and high volume points towards Triton or similar engines.

02 Platform

Existing Kubernetes favours KServe. None favours managed services.

03 Team skills

Python only teams move fastest with BentoML or Ray Serve.

04 Ecosystem

MLflow users can start with its built in serving.

05 Prove it

Benchmark two finalists on your real model and traffic.

A plain FastAPI service remains a perfectly valid choice for a few light models. Adopt a framework when it removes real pain.

SECTION 6 · SERVING FRAMEWORKS

Framework selection pitfalls

PITFALL	BETTER PRACTICE
⚠ Choosing the most powerful tool for one small model.	✓ Match tool complexity to the size and number of models.
⚠ Ignoring licence and maintenance status.	✓ Review licences, release activity and support options first.
⚠ Trusting vendor benchmarks alone.	✓ Benchmark with your own model, payloads and hardware.
⚠ Locking business logic into framework specific code.	✓ Keep core prediction logic in plain, testable modules.

Section recap

KEY TAKEAWAYS

- Model servers provide batching, multi model hosting and standard protocols
- Triton excels at GPU throughput across frameworks
- KServe and Seldon bring serving to Kubernetes
- BentoML and Ray Serve suit Python first teams
- MLflow links registry and serving closely
- Always benchmark candidates on real workloads

TRY THIS

Serve the same model with FastAPI and with one framework from this section. Compare latency, throughput and effort.

NEXT UP

Section 7: orchestrating model services with Kubernetes.

SECTION 7

07

Kubernetes and orchestration

Running many model services reliably across a fleet of machines without babysitting each one.

IN THIS SECTION

- Why orchestration
- Core Kubernetes objects
- Resources and GPUs
- Autoscaling
- Configuration and secrets
- Packaging with Helm

SECTION 7 · ORCHESTRATION

Why orchestration?



One container is easy. Fifty containers across twenty machines, restarting and scaling on their own, needs an orchestrator.

Self healing

Failed containers restart automatically and move away from unhealthy machines.

Scaling

Replicas grow and shrink with traffic, and nodes can be added as demand rises.

Declarative

You describe the desired state and the system works continuously to match it.

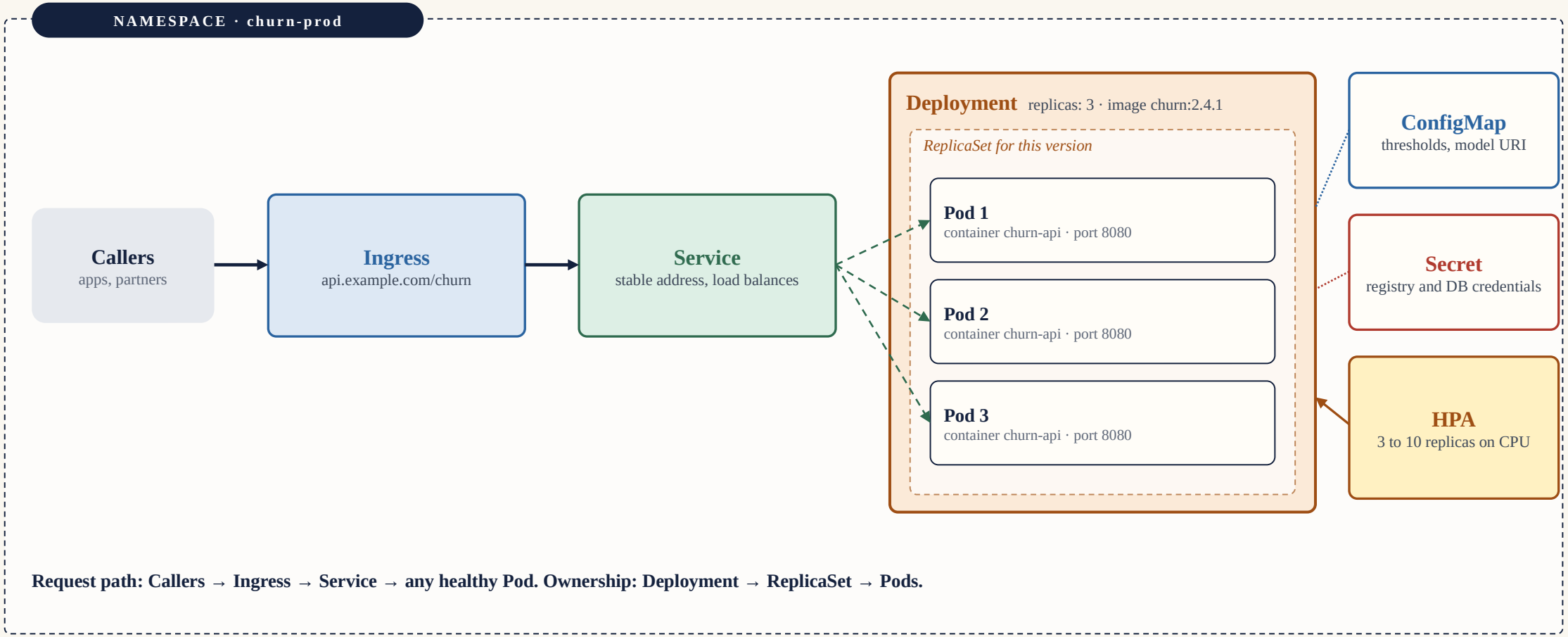
Core Kubernetes objects for model serving

Pod	Smallest unit. One or more containers sharing network and storage.
Deployment	Manages replicas of a Pod and rolls out new versions safely.
Service	Stable internal address that load balances across matching Pods.
Ingress or Gateway	Routes external HTTP traffic to Services by host and path.
ConfigMap and Secret	Inject configuration and credentials without rebuilding images.
Namespace	Groups and isolates resources for a team, model or environment.

Mental model

A Deployment keeps the right number of Pods alive. A Service gives them one stable address. An Ingress lets the outside world reach that address.

How the Kubernetes objects fit together for one model service



Request path: Callers → Ingress → Service → any healthy Pod. Ownership: Deployment → ReplicaSet → Pods.

A Deployment for a model service

churn_deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: churn
spec:
  replicas: 3
  template:
    spec:
      containers:
        name: api
        image: registry.example.com/churn:3.0.1
        resources:
          requests: {cpu: 500m, memory: 1Gi}
          limits: {memory: 2Gi}
        readinessProbe: {httpGet: {path: /ready, port: 8080}}
```

Simplified

Selectors, labels and the list marker before name are trimmed so the structure stays readable.

Pinned image

A precise version tag means every replica runs the same code and model.

Readiness

Traffic only arrives once the model is loaded and the ready check passes.

Requests and limits

Requests tell the scheduler how much CPU and memory a Pod needs. Limits cap what it may consume. Both are essential for stable model services.

Requests

Guarantee capacity and drive placement. Set them from measured usage under realistic load.

Memory limits

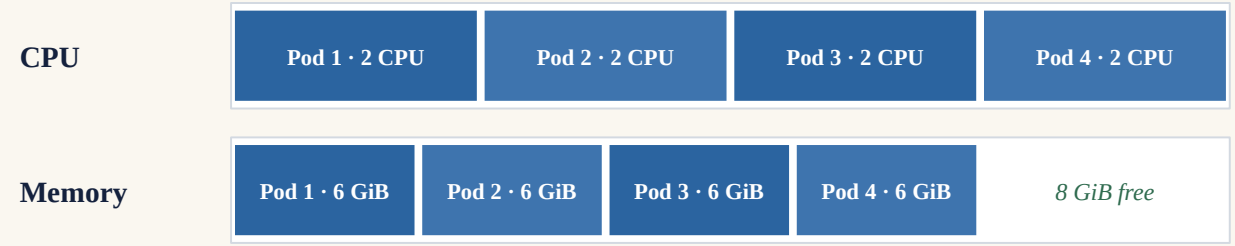
A container exceeding its memory limit is killed. Leave headroom for model loading peaks.

CPU limits

Exceeding CPU limits causes throttling and latency spikes, so many teams set requests only.

Requests decide where Pods land; limits decide how they fail

ONE NODE · 8 CPU · 32 GIB ALLOCATABLE



Pod 5 · Pending

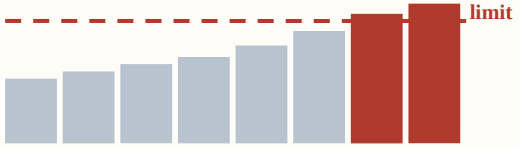
```
0/1 nodes are available: 1 Insufficient cpu.
```

Each Pod requests 2 CPU and 6 GiB. CPU runs out first, so memory sits idle. Right-size requests from measured usage.

CROSSING A LIMIT AT RUNTIME

Memory over limit

model + traffic peak



Container is killed: OOMKilled, exit code 137. Kubernetes restarts it and requests in flight fail. Leave headroom for model loading.

CPU over limit

inference burst



Container keeps running but is throttled. Latency spikes rather than errors, so many teams set CPU requests without limits.

! Set requests from measured p95 usage under realistic load; set the memory limit above the loading peak.

SECTION 7 · ORCHESTRATION

Scheduling GPUs

GPUs are exposed to Kubernetes through device plugins, then requested by Pods like any other resource. They are expensive, so placement deserves care.

Request GPUs

Pods ask for a whole number of GPUs. A device plugin makes them visible on each node.

Dedicated pools

Taints and tolerations keep ordinary workloads off costly GPU nodes.

Sharing

Techniques such as multi instance GPU or time slicing let small models share one card.

SECTION 7 · ORCHESTRATION

Scaling Pods versus scaling nodes

PODS

Horizontal Pod Autoscaler

- Adds or removes replicas of a Deployment
- Reacts to CPU, memory or custom metrics
- Fast, typically within a minute
- Limited by free capacity on nodes

NODES

Cluster or node autoscaler

- Adds or removes whole machines
- Reacts to Pods that cannot be scheduled
- Slower, often several minutes
- Tools include Cluster Autoscaler and Karpenter

VERDICT Use both together. Pods scale for quick bursts, nodes scale to supply capacity, and event based tools such as KEDA scale on queue length.

SECTION 7 · ORCHESTRATION

Choosing the right autoscaling signal

1 CPU utilisation

Simple default for CPU bound models with steady request sizes.

2 Requests per second

Better when request cost is predictable and traffic is spiky.

3 Queue length

Ideal for asynchronous and batch style workers reading from a queue.

4 Latency percentile

Scale when slow responses appear, with care to avoid oscillation.

5 GPU utilisation

Useful for GPU servers, exported through vendor monitoring tools.

6 Scheduled scaling

Pre scale before known peaks such as morning logins or match days.

The autoscaler's arithmetic, and a day of replicas

HPA FORMULA

desired = ⌈ current × observed ÷ target ⌉

Morning surge · target CPU 60%
4 replicas at 90%: 4 × 90 ÷ 60 = 6

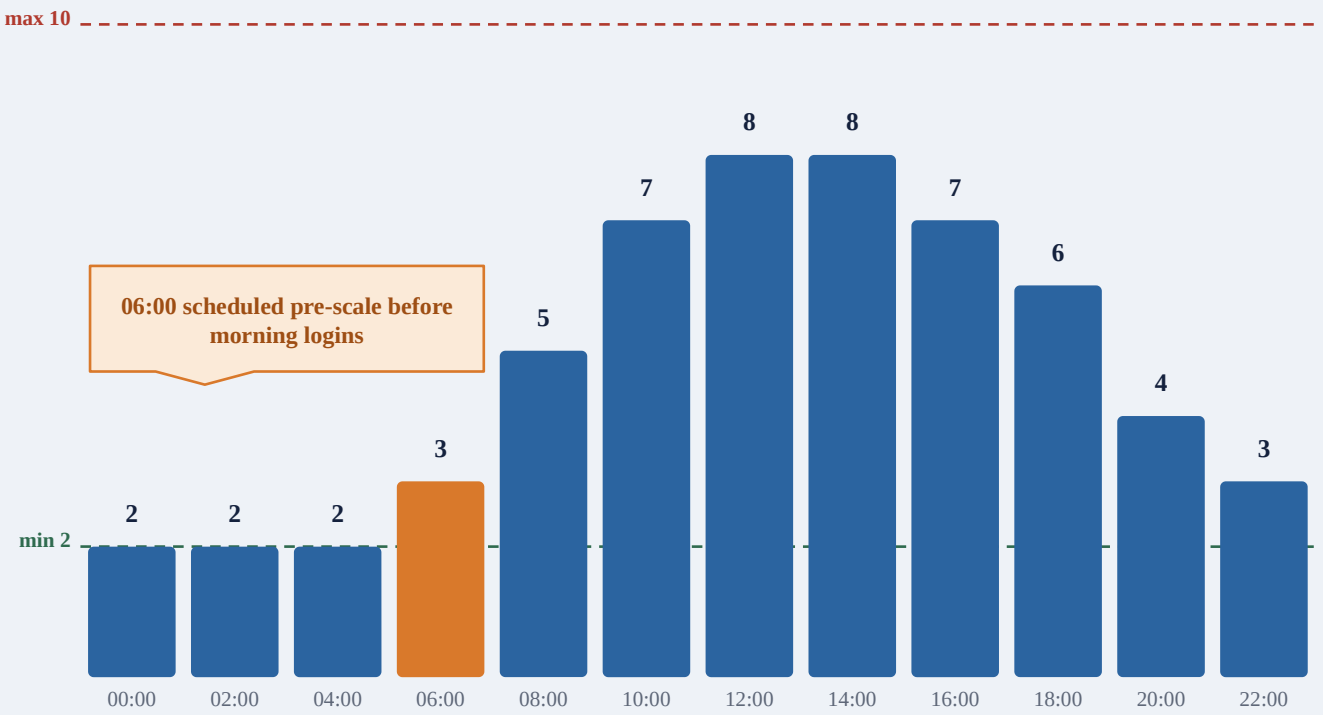
6 Pods

Evening lull
6 replicas at 45%: 6 × 45 ÷ 60 = 4.5

5 Pods

Always rounded up. Scale-down waits a 5 minute stabilisation window by default.

REPLICAS THROUGH ONE DAY · MIN 2, MAX 10



! Pick the signal that tracks user pain: CPU for steady requests, queue length for workers, requests per second for spiky APIs.

SECTION 7 · ORCHESTRATION

Configuration and secrets

Keep images identical across environments and inject everything that differs, such as model version, endpoints and credentials, at deploy time.

ConfigMaps

Hold non sensitive settings like thresholds, feature flags and the active model version.

Secrets

Hold credentials. Enable encryption at rest and restrict who can read them.

External stores

Sync from Vault or a cloud secret manager so rotation happens in one place.

SECTION 7 · ORCHESTRATION

Packaging with Helm

Helm bundles Kubernetes manifests into reusable, parameterised charts, so every model service can share one well tested template.

Charts

A chart is a folder of templates plus default values describing a full application.

Values

Each model or environment overrides a few values such as image tag and replica count.

Releases

Helm tracks installed versions and can roll back to a previous release quickly.

SECTION 7 · ORCHESTRATION

Kubernetes pitfalls for ML

PITFALL	BETTER PRACTICE
⚠ No resource requests, so Pods land on overloaded nodes.	→ ✓ Set requests from load tests and review them regularly.
⚠ Readiness that passes before the model is loaded.	→ ✓ Make readiness depend on a successful warm up prediction.
⚠ One giant namespace for everything.	→ ✓ Separate namespaces per team or environment with quotas.
⚠ Hand edited manifests drifting between clusters.	→ ✓ Template with Helm and deploy through GitOps.

Section recap

KEY TAKEAWAYS

- Orchestrators restart, place and scale containers for you
- Deployments, Services and Ingress form the core serving trio
- Requests and limits keep model services stable
- Scale Pods quickly and nodes for capacity
- Inject configuration and secrets at deploy time
- Template services with Helm charts

TRY THIS

Deploy your container to a local cluster such as kind or minikube with two replicas, a readiness probe and an autoscaler.

NEXT UP

Section 8: managed cloud platforms for machine learning.

SECTION 8

08

Cloud and managed platforms

Letting a provider run the undifferentiated heavy lifting so your team can focus on models and outcomes.

IN THIS SECTION

- The managed spectrum
- Amazon SageMaker
- Google Vertex AI
- Azure Machine Learning
- Serverless containers
- Managed versus self hosted

The spectrum of hosting choices

Fully managed endpoints	Upload a model, receive an endpoint. Minimal operations work.
Serverless containers	Bring a container, the platform scales it and handles networking.
Managed Kubernetes	Provider runs the control plane, you run workloads and nodes.
Virtual machines	You manage the operating system, runtime and scaling.
On premises hardware	Full control and full responsibility, including hardware and power.

Rule of thumb

Move down the stack only when you have a specific need the higher option cannot meet, such as cost at scale, special hardware or strict control.

Who runs what: the hosting spectrum as a responsibility map

	Fully managed endpoint	Serverless containers	Managed Kubernetes	Virtual machines	On premises
Model and inference code	You	You	You	You	You
Container and serving runtime	Provider	You	You	You	You
Scaling and load balancing	Provider	Provider	You	You	You
Operating system and patching	Provider	Provider	Shared	You	You
Cluster control plane	Provider	Provider	Provider	You	You
Hardware, power, network	Provider	Provider	Provider	Provider	You

Count the orange cells: each one needs an owner, an on-call rota and a patching plan.



Amazon SageMaker

SageMaker is the AWS platform for building, training and deploying models, with several inference options for different traffic shapes.

Real time endpoints

Persistent, autoscaling endpoints for low latency requests, including multi model hosting.

Serverless and async

Serverless inference for spiky traffic, asynchronous inference for large or slow requests.

Batch transform

Runs a model over datasets in storage without keeping an endpoint running.

Google Vertex AI

Vertex AI unifies the Google Cloud machine learning workflow, from data and training through a model registry to online and batch prediction.

Online prediction

Deploy registered models to autoscaling endpoints with traffic splitting between versions.

Batch prediction

Score data from BigQuery or Cloud Storage on demand or on a schedule.

Pipelines

Orchestrate training and deployment steps with managed pipeline runs.

Azure Machine Learning

Azure Machine Learning provides workspaces, registries and managed endpoints, integrating with the wider Microsoft data and security ecosystem.

Managed online endpoints

Host models with blue green deployments and traffic allocation across versions.

Batch endpoints

Run large scoring jobs over data in storage using compute clusters.

Governance

Ties into Azure identity, networking and policy controls for enterprises.

SECTION 8 · CLOUD PLATFORMS

Serverless containers and functions

Services such as Google Cloud Run, AWS Lambda, AWS App Runner and Azure Container Apps run containers or functions without managing servers.

Strengths

Scale to zero, pay per use, simple deployment and automatic HTTPS endpoints.

Limits

Cold starts, request time limits, memory ceilings and varying GPU availability.

Sweet spot

Light models, internal tools, prototypes and unpredictable traffic patterns.

SECTION 8 · CLOUD PLATFORMS

Managed versus self hosted

MANAGED

Provider operates the platform

- Fast start with little infrastructure skill
- Built in scaling, security and monitoring
- Higher unit cost at large steady volume
- Some lock in to provider interfaces

SELF HOSTED

Your team operates the platform

- Full control of hardware and software
- Potentially cheaper at large, steady scale
- Needs dedicated platform engineers
- You own uptime, patches and upgrades

VERDICT Default to managed until cost, control or compliance give a measured reason to self host. Revisit the decision yearly.

SECTION 8 · CLOUD PLATFORMS

Controlling cloud inference cost

1 Right size instances

Measure real utilisation and pick the smallest instance that meets latency targets.

2 Scale to demand

Use autoscaling and scale to zero for quiet endpoints.

3 Spot capacity

Run batch scoring on discounted spare capacity that may be interrupted.

4 Commit for baseline

Use savings plans or reservations for predictable steady load.

5 Share endpoints

Host several small models on one endpoint where supported.

6 Tag everything

Label resources by team and model so costs are attributable.

SECTION 8 · CLOUD PLATFORMS

Three ways to host one model, priced for a month

Always on, sized for peak

`3 replicas × 730 h × £1.50`

£3,285

per month

Simple, but pays for idle GPUs every night.

£0.33 per 1,000 predictions

Autoscaled 1 to 3

`avg 1.6 replicas × 730 h × £1.50`

£1,752

per month

47% cheaper; needs warm-up and good signals.

£0.18 per 1,000 predictions

Nightly batch on spot

`2 h × 30 nights × £0.60`

£36

per month

Only if predictions can be a day old.

£0.004 per 1,000 predictions

Assumes 10 million predictions a month, an illustrative £1.50 per GPU hour and a 60% spot discount.

LEVER ORDER Right-size first, then autoscale, then move anything that tolerates delay to batch on spot, then commit to a baseline.

Tag every endpoint by team and model, or none of these numbers can be measured.

Cloud pitfalls

PITFALL	BETTER PRACTICE
Leaving test endpoints running for months.	Set budgets, alerts and automatic clean up for idle endpoints.
Moving large datasets across regions repeatedly.	Keep compute close to data and watch egress charges.
Clicking resources together in the console.	Define infrastructure as code with Terraform or similar tools.
Ignoring provider quotas until launch day.	Request GPU and endpoint quota increases well in advance.

SECTION 8 · CLOUD PLATFORMS

Section recap

KEY TAKEAWAYS

- Hosting options range from fully managed to on premises
- SageMaker, Vertex AI and Azure ML cover online, batch and pipelines
- Serverless containers suit light and spiky workloads
- Default to managed, self host for measured reasons
- Cost control is a design activity, not an afterthought

TRY THIS

Price the same model on one managed endpoint and one serverless container for 10,000 and 1,000,000 requests a day using the provider calculators.

NEXT UP

Section 9: automating delivery with CI and CD for machine learning.

SECTION 9

09

CI and CD for machine learning

Automating the journey from a commit or new data to a safely released model.

IN THIS SECTION

- What changes in ML pipelines
- Pipeline stages
- Model registry
- Quality gates
- Infrastructure as code and GitOps
- Environments and approvals

What makes ML delivery different?

Software changes when code changes. ML systems change when code, data or the model change, so all three need pipelines.

Code

Serving code, feature logic and training scripts go through normal software CI.

Data

New data needs validation for schema, quality and distribution before training.

Model

Each new model needs evaluation, comparison and approval before release.

A complete ML delivery pipeline

01 — Commit

Code or config change triggers CI with linting and unit tests.

02 — Train

Pipeline validates data, trains and evaluates the candidate model.

03 — Register

Passing models are stored with metrics and lineage in the registry.

04 — Build

Service image is built, scanned and pushed with the model version.

05 — Release

Staged rollout to test, then production, with automated checks.

Separate continuous integration of code, continuous training of models and continuous delivery of services, but connect them through the registry.

SECTION 9 · CI AND CD

The model registry

A model registry is the single source of truth for which model versions exist, how they performed and which one each environment should serve.

Stores

Artifacts, metrics, parameters, data references, code commit and model card for every version.

Stages

Marks versions as candidate, staging, production or archived, often through aliases.

Examples

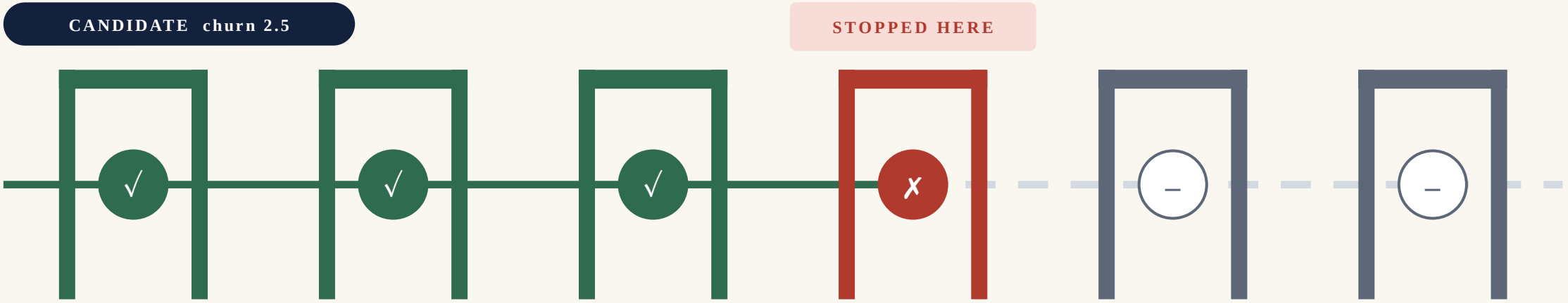
MLflow Model Registry, SageMaker Model Registry, Vertex AI Model Registry and Azure ML registries.

Quality gates before release

Gate	What it checks	Example rule
Unit tests	Feature and serving code correctness	All tests pass
Data validation	Schema, missing values, ranges	No new nulls in key fields
Model quality	Metric on a fixed holdout set	Not worse than production model
Fairness	Metric gaps across groups	Gap within agreed tolerance
Performance	Latency and memory under load	p95 latency under budget
Security	Image and dependency scans	No critical vulnerabilities

Gates must be automated and visible. A gate that a person can quietly skip is a suggestion, not a gate.

A candidate model meets the quality gates, and is stopped at one



WHAT HAPPENS NEXT

The pipeline fails, the owner gets the slice report, and nothing reaches the registry. Overriding needs a recorded approval, not a quiet skip.

NOTICE

Better overall AUC did not save it. Gates check what averages hide.

Infrastructure as code

1 Declare it

Describe clusters, endpoints, buckets and permissions in versioned files.

2 Review it

Infrastructure changes go through pull requests like application code.

3 Reproduce it

Recreate entire environments for testing or disaster recovery.

4 Tools

Terraform, OpenTofu, Pulumi, AWS CDK and cloud native templates.

5 Plan before apply

Preview changes and require approval for production.

6 Detect drift

Regularly compare real infrastructure against the declared state.

GitOps

DEFINITION

GitOps is a delivery approach where a Git repository holds the desired state of every environment, and an automated agent continuously makes the live system match it.

EXAMPLE

To release model 3.1, an engineer merges a change that updates the image tag in the production values file. Argo CD notices and rolls out the new version automatically.

REMEMBER

If it is not in Git, it is not deployed. Rollback is simply reverting a commit.

Environments and promotion

1. **Development.** Fast feedback for engineers with small samples and mocked dependencies.
2. **Integration.** Services talk to real dependencies and contract tests run automatically.
3. **Staging.** Production like data volumes, load tests and end to end checks.
4. **Production canary.** A small share of real traffic validates the release.
5. **Production.** Full rollout once canary metrics remain healthy.
6. **Promotion rule.** The same image and model version move forward unchanged at every stage.

SECTION 9 · CI AND CD

Human approvals where they matter

Automation handles the routine. Humans should approve the decisions where context, risk or accountability matter most.

When to require

High impact decisions, regulated domains, large metric changes or first releases of new models.

What to show

Metric comparison, fairness results, data changes and a link to the model card.

Record it

Log who approved, when and why, so audits can reconstruct every release.

CI and CD pitfalls

PITFALL	BETTER PRACTICE
Training models on a laptop and uploading files manually.	Train in a pipeline that records data, code and parameters.
Comparing a new model on a different test set each time.	Keep a versioned evaluation set and a fixed comparison procedure.
Releasing straight to all users.	Promote through staging and a canary with automatic checks.
Pipelines nobody can read.	Keep pipeline definitions in Git with clear stage names and owners.

Section recap

KEY TAKEAWAYS

- ML pipelines must handle code, data and model changes
- The registry connects training and delivery
- Automated gates protect quality, fairness and security
- Infrastructure as code and GitOps make releases reproducible
- Promote the same artifact through every environment
- Use human approval for high risk decisions

TRY THIS

Draw your ideal pipeline for one model. Mark every gate, the tool that enforces it and who is alerted when it fails.

NEXT UP

Section 10: testing models for production.

SECTION 10

10

Testing models for production

Building confidence in code, data and model behaviour before real users ever see a prediction.

IN THIS SECTION

- The ML testing pyramid
- Data tests
- Model behaviour tests
- Integration and contract tests
- Load and soak tests
- Testing in production safely

The ML testing pyramid

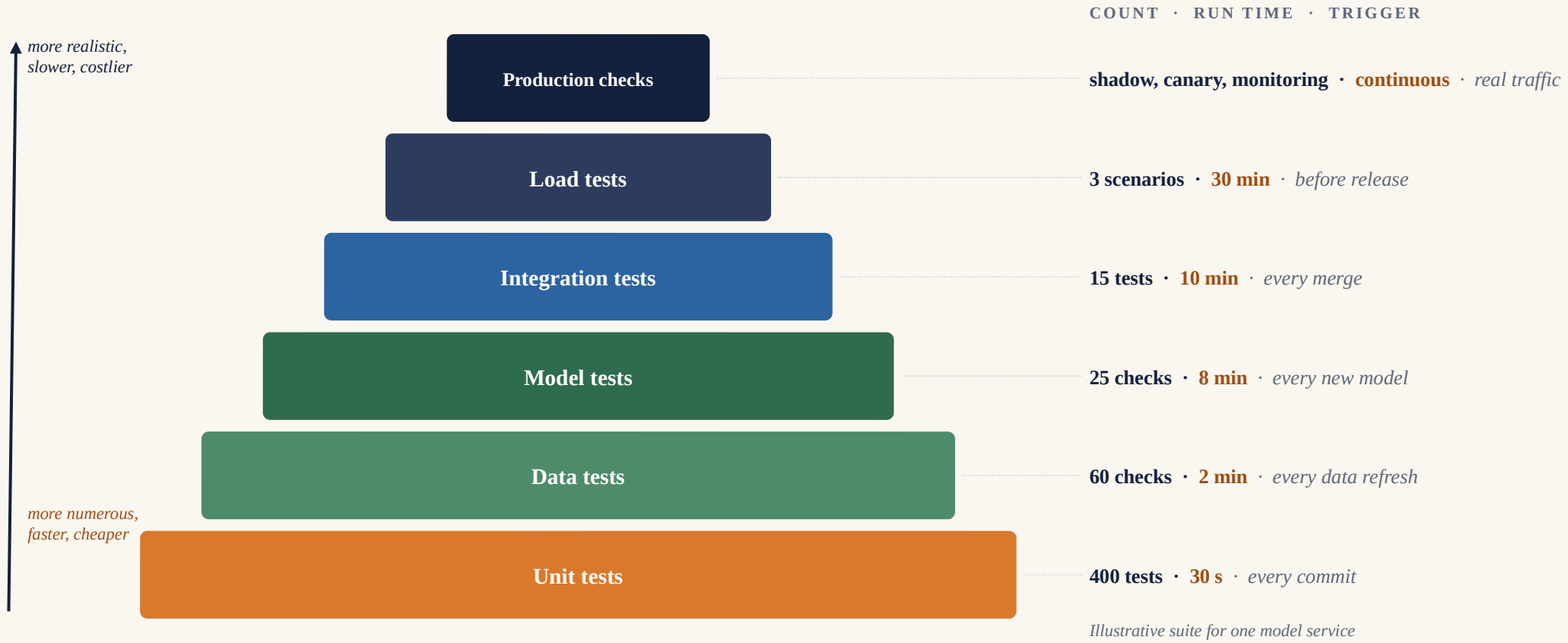
Unit tests	Feature functions, parsers and utilities tested in isolation.
Data tests	Schemas, ranges, nulls and distributions of training and serving data.
Model tests	Quality thresholds, behaviour checks and fairness on fixed datasets.
Integration tests	API, model and dependencies working together in a test environment.
Load tests	Latency, throughput and stability under realistic and peak traffic.
Production checks	Shadow, canary and monitoring validate behaviour with real data.



Balance

Many fast, cheap tests at the bottom, fewer slow and expensive tests at the top. Every layer catches problems the others miss.

The ML testing pyramid with counts, run times and triggers



! Push each check as low as it will go: a bug caught by a 30 second unit test never reaches a 30 minute load test.

Data tests worth automating

1 Schema
Expected columns and types are present, with no unexpected additions.

2 Completeness
Missing value rates stay below agreed limits for key fields.

3 Ranges
Values sit within plausible bounds, such as ages and prices.

4 Uniqueness
Identifiers that should be unique really are unique.

5 Freshness
Data is recent enough for the decision it supports.

6 Distribution
Feature distributions have not shifted sharply from the reference.

SECTION 10 · TESTING

Behavioural tests for models

Beyond a single accuracy number, behavioural tests check that the model reacts sensibly to deliberate, understandable changes in its inputs.

Invariance

Changing an irrelevant detail, such as a customer name, should not change the prediction.

Directional

Raising missed payments should never lower a credit risk score.

Minimum functionality

Simple, obvious cases must be predicted correctly every time.

SECTION 10 · TESTING

Behavioural tests, shown as real inputs and outputs

Test type	Input change		Risk score	Rule	Result
Invariance	Name: Sarah Jones	**Name: Mohammed Ali**	0.21 → 0.21	*must not change*	✓ PASS
Directional	Missed payments: 0	**Missed payments: 3**	0.21 → 0.48	*must not go down*	✓ PASS
Minimum functionality	Obvious high-risk case	**6 missed, debt 3× income**	0.91	*must be ≥ 0.80*	✓ PASS
Directional	Income: £30,000	**Income: £60,000**	0.30 → 0.36	*must not go up*	✗ FAIL

CAUGHT Higher income should never raise credit risk. The model had learned a quirk of the data, and CI blocked it before any customer saw it.

A behavioural test with pytest

test_model_behaviour.py

```
import joblib, pandas as pd

model = joblib.load("churn_model_v3.joblib")

def score(**fields):
    return model.predict_proba(pd.DataFrame([fields]))[0][1]

def test_longer_tenure_does_not_raise_churn():
    short = score(age=40, tenure=2, plan="basic")
    long = score(age=40, tenure=60, plan="basic")
    assert long <= short

def test_unknown_plan_is_handled():
    assert 0 <= score(age=40, tenure=10, plan="new") <= 1
```

Directional

Encodes domain knowledge: loyal customers should not look riskier.

Robustness

A plan never seen in training must still produce a valid probability.

Run in CI

These tests run on every candidate model before registration.

Integration and contract tests

Integration tests run the real service with its dependencies. Contract tests make sure the API still honours what every client expects.

Integration

Start the container, call the endpoint with sample payloads and check status codes and response shape.

Contract

Store example requests and responses agreed with consumers and test them on every build.

Golden records

Keep a small set of inputs with known expected outputs to detect silent changes.

Performance test types

Test	Purpose	Typical question answered
Load	Expected traffic levels	Do we meet latency targets at normal peak?
Stress	Beyond expected limits	Where and how does the service break?
Spike	Sudden bursts	Does autoscaling react fast enough?
Soak	Long duration	Do memory leaks or slowdowns appear over hours?
Cold start	Fresh instances	How long until a new replica serves traffic?

Tools such as Locust, k6 and JMeter generate traffic. Always test with realistic payload sizes and feature values.

SECTION 10 · TESTING

Testing in production, safely

Some behaviour only appears with real traffic. Safe techniques let you learn from production without exposing every user to risk.

Shadow traffic

Copy real requests to the new model and compare outputs without returning them to users.

Canary

Send a small share of traffic to the new version and watch error and quality signals.

Synthetic probes

Call endpoints continuously with known inputs to detect outages and drift in outputs.

Testing pitfalls

PITFALL	BETTER PRACTICE
⚠ Only checking overall accuracy.	✓ Add behavioural, slice and fairness tests to every evaluation.
⚠ Load testing with tiny dummy payloads.	✓ Use realistic payloads and feature distributions.
⚠ Tests that depend on random training runs.	✓ Fix seeds and test against stored artifacts.
⚠ No tests for the preprocessing code.	✓ Unit test every feature transformation and edge case.

SECTION 10 · TESTING

Knowledge check

? A new model improves overall AUC but performs much worse for customers in one region. Should it ship?

MODEL ANSWER

Not automatically. Slice tests have revealed a regression for an important group. The team should investigate the cause, discuss business and fairness impact with stakeholders, and either fix the model, add a fallback for that region or make an explicit, documented decision.

WHY IT MATTERS

Averages hide harm. Slice metrics protect both users and the business.

Section recap

KEY TAKEAWAYS

- Test code, data, model behaviour and the running service
- Automate data tests for schema, ranges and freshness
- Behavioural tests encode domain knowledge
- Contract tests protect API consumers
- Load, spike and soak tests reveal operational limits
- Shadow and canary release test safely in production

TRY THIS

Write three behavioural tests and one contract test for your model service and add them to your CI pipeline.

NEXT UP

Section 11: release strategies for getting new models live safely.

SECTION 11

11

Release strategies

How to move users from one model version to the next while keeping risk small and reversible.

IN THIS SECTION

- Recreate and rolling
- Blue green
- Canary releases
- Shadow deployment
- A/B tests and bandits
- Feature flags and rollback

Recreate versus rolling update

RECREATE

Stop everything, then start

- All old replicas stop before new ones start
- Simple and avoids mixed versions
- Causes downtime during the switch
- Acceptable for internal batch services

ROLLING

Replace replicas gradually

- New replicas start as old ones drain
- No downtime when readiness is correct
- Two versions serve traffic briefly
- Default strategy for Kubernetes Deployments

VERDICT Rolling updates suit most online services. Make sure both versions are compatible with the same API contract during the overlap.

Recreate, rolling and blue green, drawn on the same clock



Downtime: recreate only. Mixed versions: rolling. Double capacity: blue green.

Blue green deployment

Blue green runs two complete environments. Blue serves users while green receives the new version, then traffic switches in one step.

How it works

Deploy and test green fully, then point the router or load balancer from blue to green.

Strength

Instant cutover and instant rollback by switching back to blue.

Cost

Requires double capacity during the release window.

SECTION 11 · RELEASE STRATEGIES

Canary release in practice

01 — Deploy
Start the new version beside the old one with no traffic.

02 — One percent
Route a small slice of real requests to the canary.

03 — Observe
Compare errors, latency and prediction quality signals.

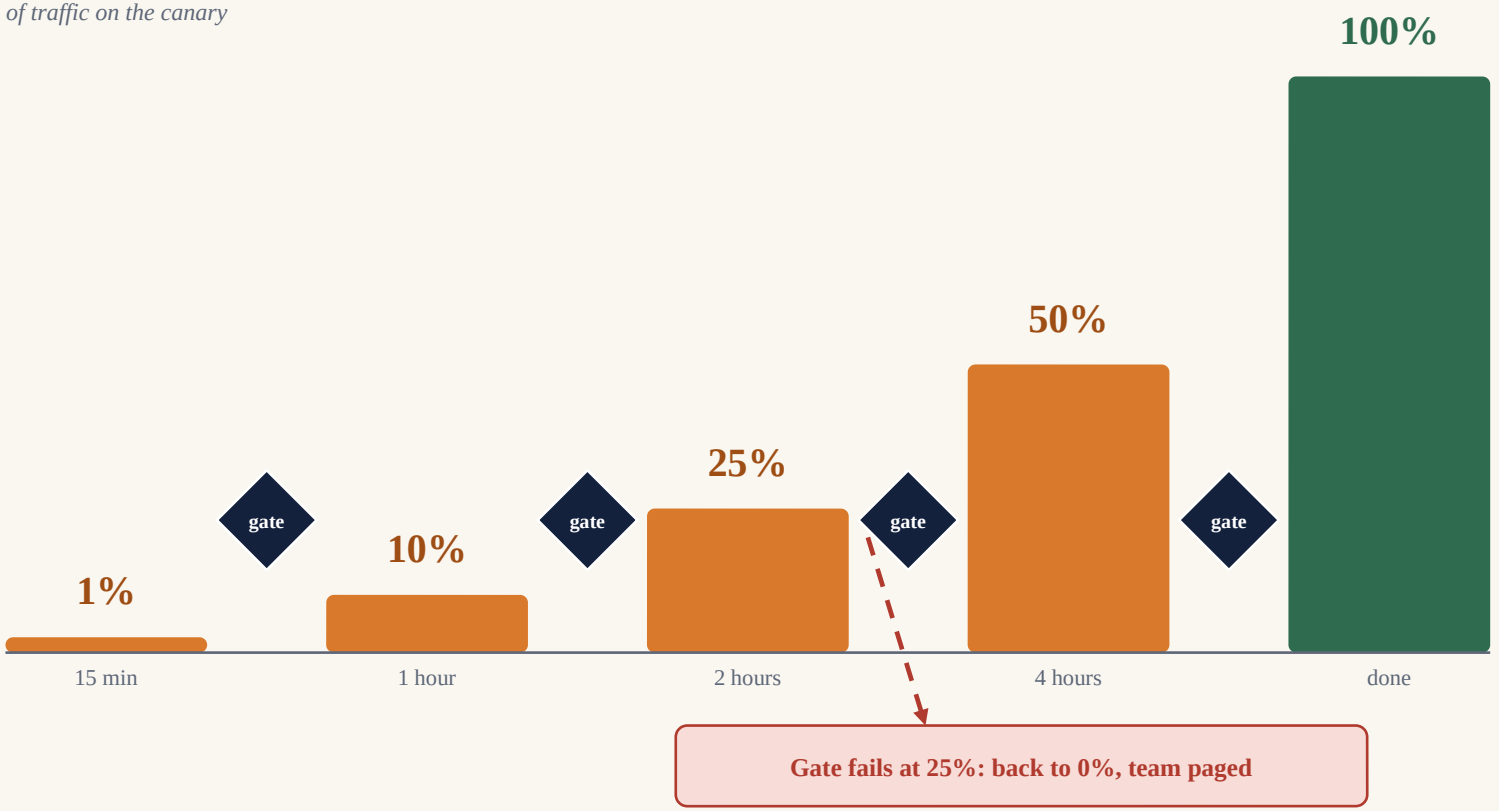
04 — Expand
Increase gradually, for example to 10, 25, 50 percent.

05 — Complete
Shift all traffic, or roll back automatically on bad signals.

Tools such as Argo Rollouts, Flagger, service meshes and managed endpoint traffic splitting automate these steps and the rollback decision.

A canary climbing its staircase, with an automatic way down

share of traffic on the canary



EACH GATE CHECKS

Error rate ≤ stable + 0.1 pp

p99 latency ≤ stable + 10%

Prediction mix close to stable

Business signal not worse

Example thresholds

! Judge model canaries on prediction quality too: a model can be fast and error-free while being wrong.

Shadow deployment

DEFINITION

In a shadow deployment the new model receives a copy of real production requests, but its predictions are only logged and compared, never returned to users.

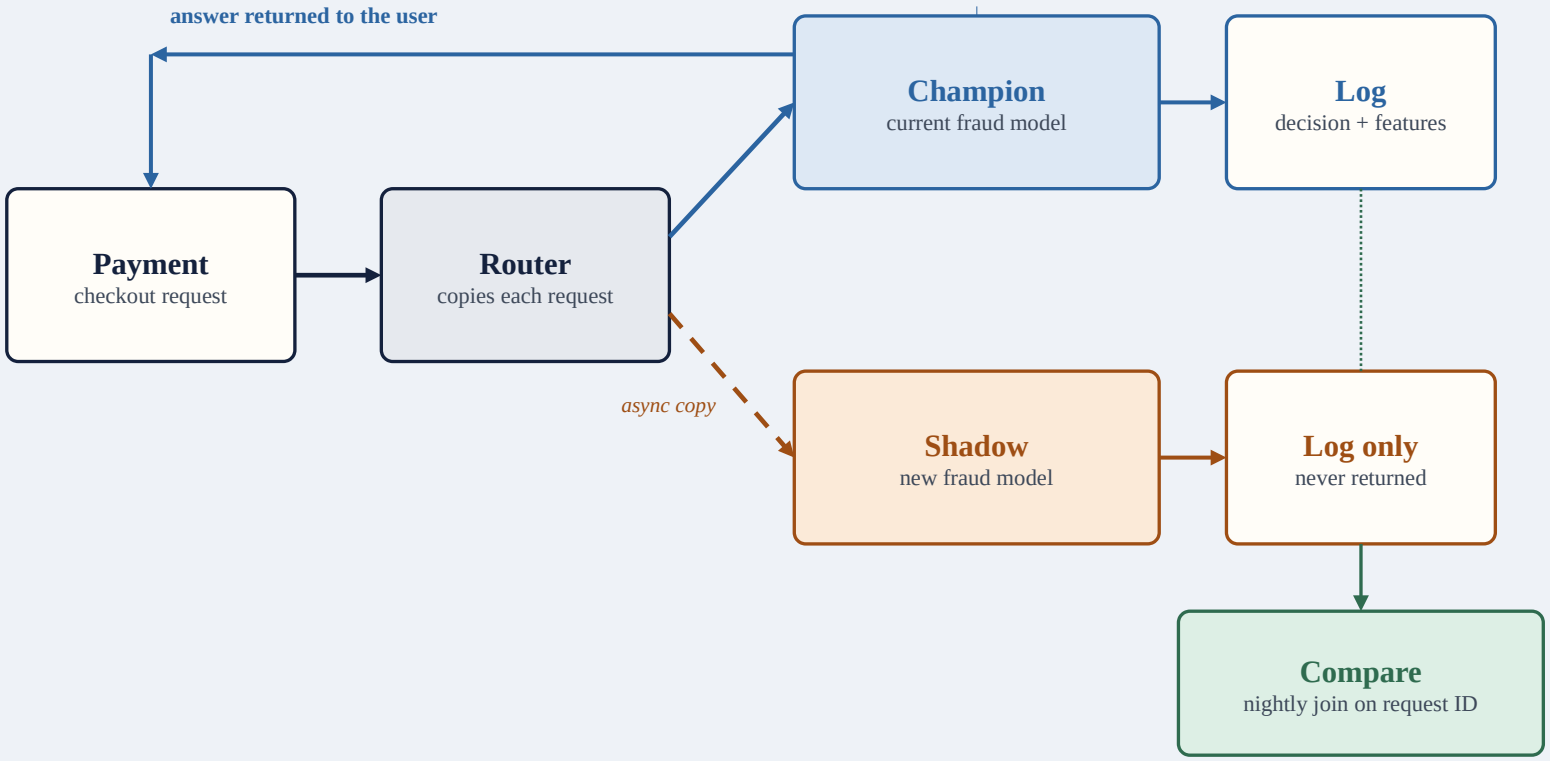
EXAMPLE

A new fraud model shadows the current one for two weeks. Analysts compare flagged transactions and confirm it catches more fraud without many extra false alarms before any customer is affected.

REMEMBER

Shadow tests behaviour on real data with zero user risk, but doubles inference cost.

Shadow deployment: real traffic, zero user risk



TWO-WEEK SHADOW REPORT

Payments compared	**1.2 million**
Same decision	**98.4%**
Flagged by champion	**1,980**
Flagged by shadow	**2,310**
Extra fraud confirmed	**+190 cases**
Shadow p99 latency	**72 ms (budget 100)**

Illustrative figures

! The shadow must never touch the user path: separate resources, async copies, strict timeouts.

A/B testing models

An A/B test randomly assigns users to the current or new model and measures the effect on a business outcome, not just model metrics.

Randomise users

Assign by user, not by request, so each person has a consistent experience.

Decide upfront

Fix the primary metric, guardrail metrics and test duration before starting.

Mind the stats

Small effects need large samples. Work with an analyst on sample size and significance.

Multi armed bandits

DEFINITION

A bandit approach shifts traffic automatically towards the model that is performing better while the test runs, balancing learning with earning.

EXAMPLE

An ecommerce site tests four recommendation models. Traffic gradually concentrates on the model with the highest click through rate, reducing the cost of showing weaker models.

REMEMBER

Bandits optimise quickly but make clean causal conclusions harder than A/B tests.

Feature flags for models

Feature flags separate deploying code from releasing behaviour, letting you switch models on or off for chosen users without redeploying.

Targeting

Enable a new model for staff, a region or a percentage of users first.

Kill switch

Turn a misbehaving model off in seconds and fall back to a safe default.

Hygiene

Remove old flags once a release is complete to avoid confusing logic.

A rollback plan you can trust

1. **Keep the previous version warm.** Retain the last good image and model so switching back is immediate.
2. **Define triggers.** Write down the metric thresholds that force a rollback without debate.
3. **Automate the switch.** Let the rollout tool reverse traffic automatically when triggers fire.
4. **Mind the data.** Ensure schemas and stored predictions remain compatible with the old version.
5. **Rehearse.** Practise rollback in staging so it works under pressure.
6. **Review.** After any rollback, hold a blameless review and record lessons.

Choosing a release strategy

Strategy	User risk	Extra cost	Best when
Recreate	High	None	Downtime is acceptable, internal tools
Rolling	Medium	Low	Standard updates with compatible APIs
Blue green	Low	High	Instant switch and rollback needed
Canary	Low	Low	Gradual validation with real traffic
Shadow	None	High	High stakes models need real data proof
A/B test	Medium	Medium	Business impact must be measured

Release pitfalls

PITFALL	BETTER PRACTICE
⚠ Judging a canary only on error rates.	✓ Also compare prediction distributions and business signals.
⚠ Canary windows too short to see real behaviour.	✓ Run long enough to include peak hours and varied users.
⚠ Stopping an A/B test as soon as it looks good.	✓ Fix duration and sample size before starting.
⚠ No tested path back to the old model.	✓ Keep the previous version deployable and rehearse rollback.

Section recap

KEY TAKEAWAYS

- Rolling updates avoid downtime for compatible changes
- Blue green gives instant switch and rollback at higher cost
- Canaries limit blast radius with gradual traffic
- Shadow deployments test on real data with no user exposure
- A/B tests and bandits measure business impact
- Rollback must be automatic, rehearsed and fast

TRY THIS

Design a canary plan for your model: traffic steps, duration, metrics, thresholds and who is notified.

NEXT UP

Section 12: making models faster and cheaper to serve.

SECTION 12

12

Performance optimisation

Making predictions faster and cheaper without sacrificing the quality users depend on.

IN THIS SECTION

- Latency and throughput
- Profiling first
- Quantisation, pruning, distillation
- Compilers and runtimes
- Hardware choices
- Caching and cold starts

SECTION 12 · PERFORMANCE

Latency versus throughput

LATENCY

Time for one request

- How long a single user waits
- Reported as percentiles such as p50, p95, p99
- Critical for interactive products
- Batching usually makes it slightly worse

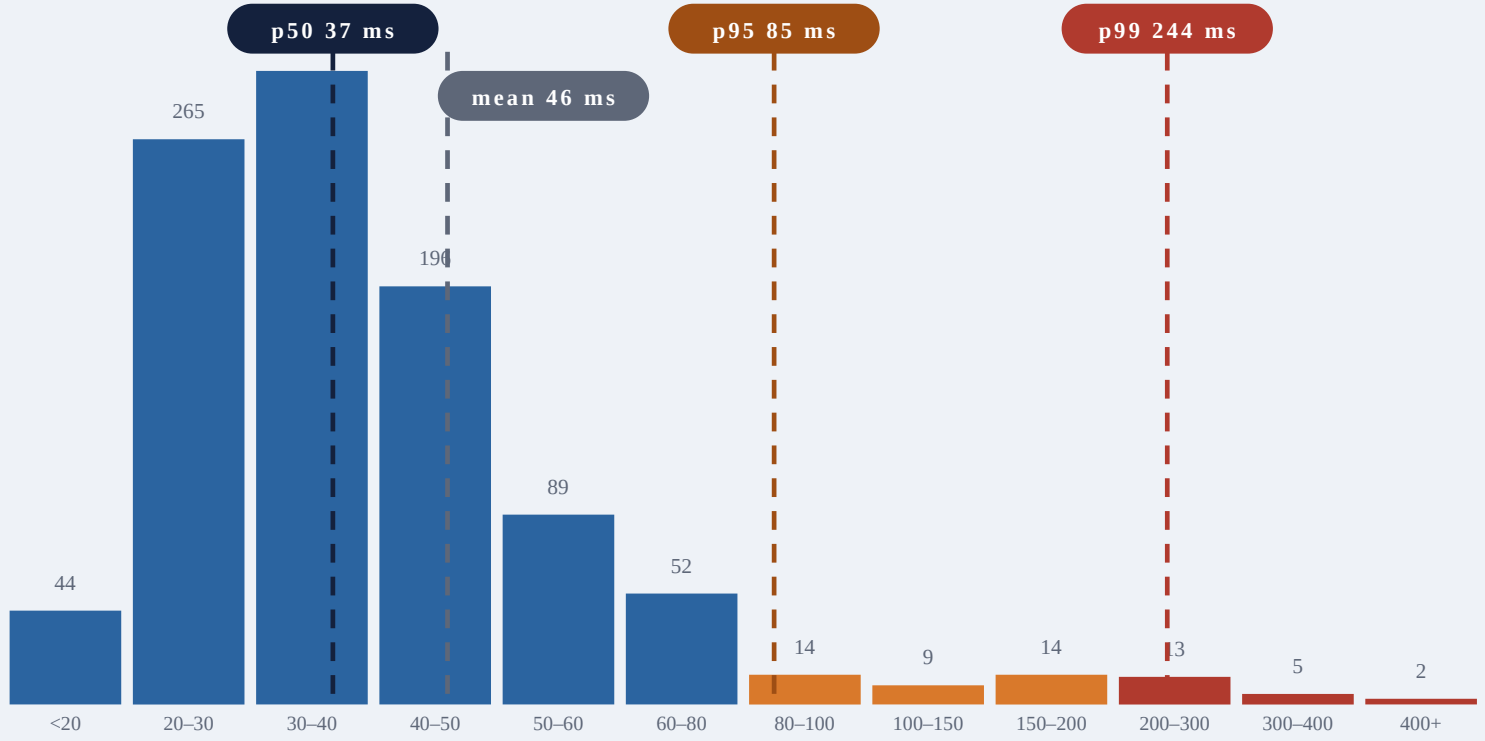
THROUGHPUT

Requests completed per second

- How much work the system handles
- Drives infrastructure cost per prediction
- Critical for batch and high volume APIs
- Batching usually improves it greatly

VERDICT Agree a latency budget at a high percentile first, then maximise throughput within it. Averages hide the slow requests users remember.

Why averages hide the slow requests users remember



Simulated latencies

THE TAIL

6.6×

p99 versus the median

FAN-OUT MAKES IT WORSE

$1 - 0.99^{20} \approx 18\%$

A page that makes 20 model calls hits at least one p99-slow call in nearly one view in five.

! Set budgets at p95 or p99, and plot the histogram before trusting the mean.

SECTION 12 · PERFORMANCE

Profile before you optimise



Guessing where time goes is usually wrong. Measure every stage of a request before changing anything.

Break it down

Time network, validation, feature lookup, preprocessing, model and serialisation separately.

Use tools

Python profilers, framework profilers and tracing systems show where time is spent.

Fix the biggest

Optimise the slowest stage first. Often it is data access, not the model.

SECTION 12 · PERFORMANCE

Quantisation

DEFINITION

Quantisation stores and computes model weights with fewer bits, for example 8 bit integers instead of 32 bit floats, reducing memory and often speeding up inference.

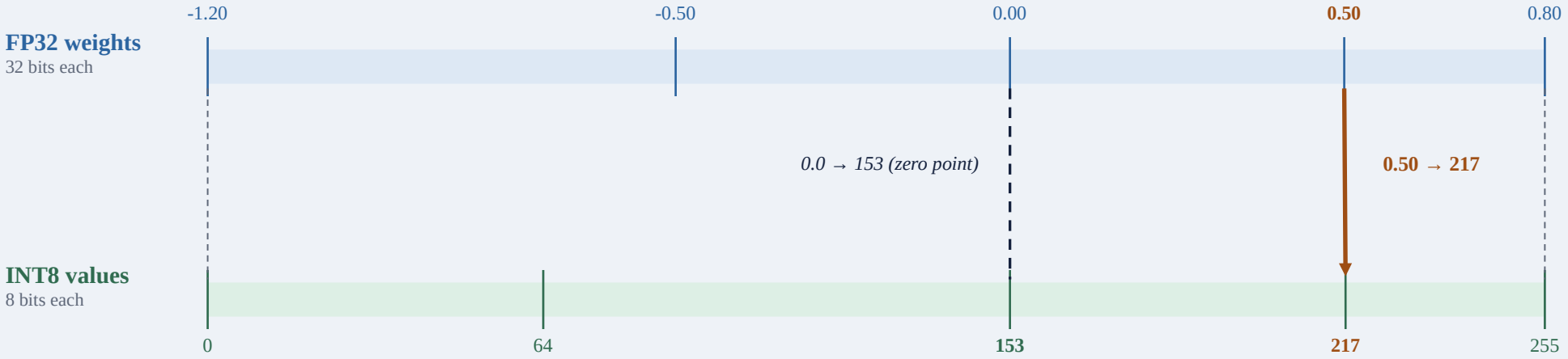
EXAMPLE

A text classifier quantised to 8 bit integers becomes about four times smaller and runs noticeably faster on CPU, with only a small, measured drop in accuracy.

REMEMBER

Always compare quality before and after. Calibrate with representative data.

Quantisation, worked out for a single weight



Scale
`(0.80 − (−1.20)) ÷ 255`

0.00784

Zero point
`round(1.20 ÷ 0.00784)`

153

Quantise 0.50
`round(0.50 ÷ 0.00784) + 153`

217

Read back
`(217 − 153) × 0.00784`

0.502 *error 0.002*

100M parameters: **400 MB** at 4 bytes → **100 MB** at 1 byte. Then re-measure accuracy on the test set.

SECTION 12 · PERFORMANCE

Model compression techniques

1 Quantisation

Fewer bits per weight and activation. Dynamic, static and quantisation aware training options exist.

2 Pruning

Removes weights or whole structures that contribute little to predictions.

3 Distillation

Trains a small student model to imitate a large teacher model.

4 Low rank methods

Approximate large weight matrices with smaller factors.

5 Smaller architectures

Choose efficient model families designed for speed from the start.

6 Feature reduction

Fewer, cheaper features speed both lookup and prediction.

Compilers and optimised runtimes

Inference engines rewrite the model graph for specific hardware, fusing operations and choosing fast kernels automatically.

ONNX Runtime

Cross platform engine with execution providers for CPUs, GPUs and accelerators.

TensorRT

NVIDIA engine that optimises models aggressively for its GPUs.

Others

OpenVINO for Intel hardware, Apache TVM and PyTorch compilation features.

Choosing inference hardware

Hardware	Strengths	Best for	Watch
CPU	Cheap, available, simple	Tree models, small networks	Slow for large deep models
GPU	Massive parallel compute	Vision, speech, LLMs, batching	Cost and idle time
Cloud accelerators	Custom silicon value	Large scale, specific frameworks	Portability limits
Edge NPUs	Low power on device	Mobile and embedded models	Operator support

Many tabular models run perfectly well on CPUs. Reach for GPUs when the model and volume justify the cost.

Caching predictions and features

If the same input appears often, the fastest prediction is the one you do not recompute. Caches also protect slow downstream systems.

Prediction cache

Store results keyed by a hash of the input with a sensible expiry time.

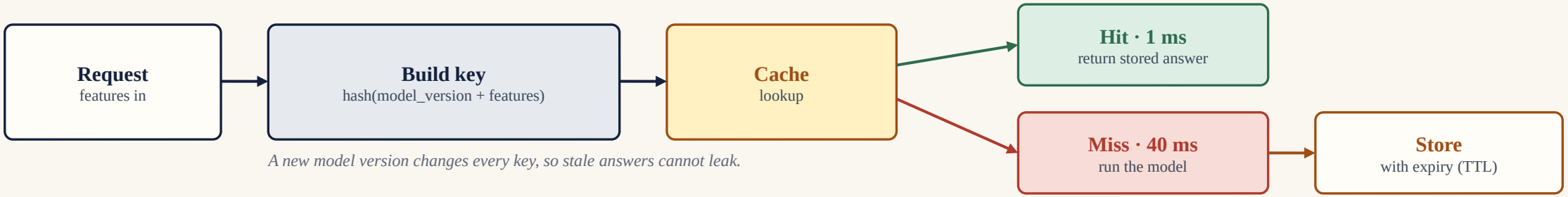
Feature cache

Keep frequently used features in memory or a fast key value store.

Invalidation

Clear caches when the model version changes so stale answers never leak.

What a prediction cache buys you, in milliseconds and calls



A new model version changes every key, so stale answers cannot leak.

AVERAGE LATENCY

$$h \times 1\ \text{ms} + (1 - h) \times 40\ \text{ms}$$

Fewer model calls also protect slow downstream systems such as the feature store.

Hit rate h	Average latency	Model calls per 1,000
0%	40 ms	1,000
50%	20.5 ms	500
80%	8.8 ms	200
95%	2.95 ms	50

! Caching only helps when inputs repeat. Measure the real hit rate before building one, and set an expiry that matches how fast features change.

SECTION 12 · PERFORMANCE

Cold starts and warm up

A new replica must load the model, initialise libraries and sometimes compile kernels before it can answer quickly.

Warm up requests

Run a few dummy predictions during startup before reporting ready.

Keep minimum replicas

Avoid scaling all the way to zero for latency sensitive services.

Faster loading

Use smaller artifacts, memory mapped weights and images cached on nodes.

SECTION 12 · PERFORMANCE

A performance tuning workflow

1. **Set budgets.** Define latency percentile targets and cost per thousand predictions.
2. **Baseline.** Load test the current service and record results.
3. **Profile.** Find the slowest stages under realistic traffic.
4. **Change one thing.** Apply one optimisation at a time to understand its effect.
5. **Check quality.** Confirm accuracy and slice metrics remain acceptable.
6. **Keep or revert.** Adopt only changes that improve the target without harm.

SECTION 12 · PERFORMANCE

Performance pitfalls

PITFALL	BETTER PRACTICE
Optimising the model while a slow database dominates latency.	Profile end to end and fix the largest bottleneck first.
Reporting only average latency.	Track p95 and p99 latency against an agreed budget.
Quantising without measuring accuracy loss.	Evaluate compressed models on the full test suite.
Benchmarking on a powerful laptop.	Benchmark on the same hardware used in production.

Section recap

KEY TAKEAWAYS

- Set latency budgets at high percentiles
- Profile every stage before optimising
- Quantisation, pruning and distillation shrink models
- Compilers and runtimes accelerate inference
- Choose hardware by model type and volume
- Cache wisely and warm up new replicas

TRY THIS

Export your model to ONNX, run it with ONNX Runtime, and compare latency and outputs with the original.

NEXT UP

Section 13: monitoring and observability.

SECTION 13

13

Monitoring and observability

Knowing, at any moment, whether your model is healthy, accurate and still delivering value.

IN THIS SECTION

- Four layers of monitoring
- Logs, metrics and traces
- Data and concept drift
- Delayed ground truth
- Alerts and objectives
- Tools and dashboards

“

Models fail silently. The service stays up while the predictions quietly become wrong.

☆ Traditional software monitoring catches crashes. ML monitoring must also catch a healthy looking service that has stopped being useful.

Four layers of ML monitoring

Infrastructure	CPU, memory, GPU, disk, network and container restarts.
Service	Request rate, error rate, latency percentiles and saturation.
Data	Input schema violations, missing values and feature distributions.
Model	Prediction distribution, confidence, and accuracy once labels arrive.
Business	Conversions, losses avoided, complaints and other outcome metrics.

Cover all four

Each layer answers a different question. Teams that watch only infrastructure discover model problems from angry customers instead of dashboards.

SECTION 13 · MONITORING

Logs, metrics and traces

1 Metrics

Numeric time series such as latency and error counts, cheap to store and ideal for alerts.

2 Logs

Detailed event records such as each request, prediction and error with context.

3 Traces

Follow one request across services to see where time is spent.

4 Prediction logs

Inputs, outputs, model version and request identifier, kept for analysis.

5 Standards

OpenTelemetry offers a vendor neutral way to emit all three signals.

6 Privacy

Mask or exclude personal data before logging, and set retention limits.

Data drift

DEFINITION

Data drift, also called covariate shift, happens when the distribution of input features in production moves away from the distribution seen during training.

EXAMPLE

A loan model trained before a recession now sees many more applicants with irregular income. The inputs look different, even though the rules linking them to default may be unchanged.

REMEMBER

Drift in inputs is an early warning, not proof that accuracy has fallen.

Concept drift

DEFINITION

Concept drift happens when the relationship between inputs and the correct answer changes, so patterns the model learned are no longer true.

EXAMPLE

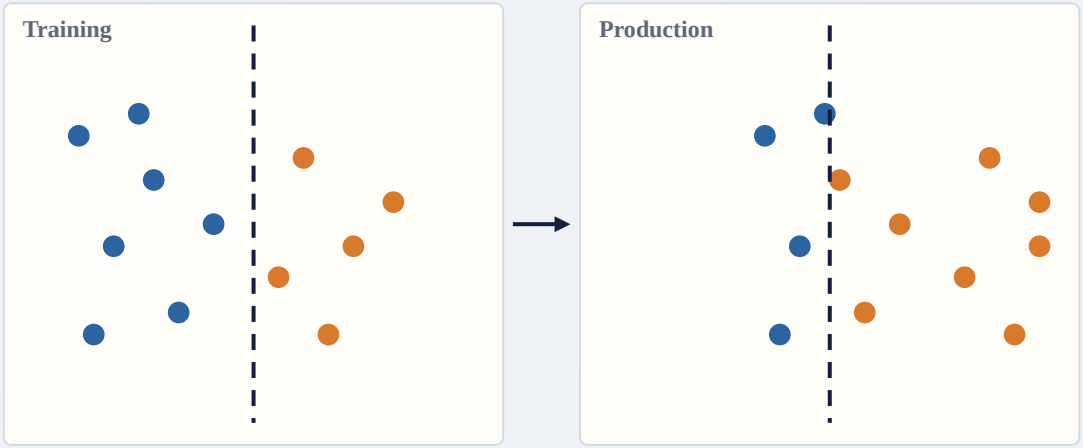
Fraudsters change tactics after a new security feature launches. Transactions look similar to before, but which ones are fraudulent has changed.

REMEMBER

Concept drift usually needs fresh labels to detect and new training data to fix.

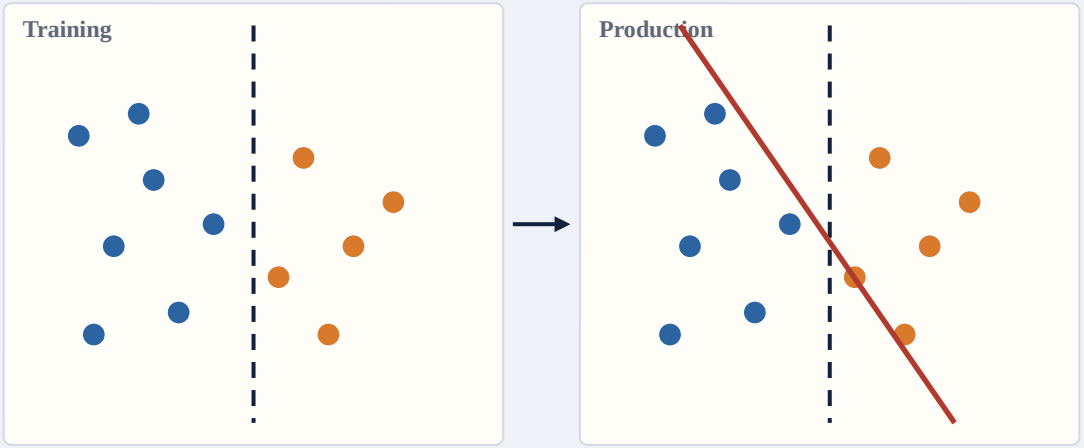
Data drift moves the points; concept drift moves the line

DATA DRIFT · COVARIATE SHIFT



Inputs move right; the rule is unchanged.

CONCEPT DRIFT



Same inputs; the true boundary (red) has rotated.

	Data drift	Concept drift
What changes	Distribution of inputs	Link between inputs and the answer
Detect without labels?	Yes: PSI, KS test, feature stats	No: needs labels or proxies
Typical response	Investigate, maybe retrain	Retrain on recent labelled data

Common drift detection approaches

Approach	Idea in plain words	Good for
Population stability index	Compares how values spread across bins	Scorecards, tabular features
Kolmogorov Smirnov test	Checks if two numeric samples differ	Continuous features
Chi squared test	Compares category frequencies	Categorical features
Distance measures	Scores how far apart distributions are	Many feature types
Domain classifier	Trains a model to tell old data from new	High dimensional data
Embedding drift	Tracks shifts in embedding space	Text, image and LLM inputs

Statistical tests become oversensitive with huge samples. Combine them with practical thresholds and human judgement.

SECTION 13 · MONITORING

When ground truth arrives late

Many outcomes are known only weeks or months later, such as loan defaults or customer churn, so accuracy cannot be measured in real time.

Proxy signals

Watch prediction distributions, confidence and early business indicators meanwhile.

Join later

Store request identifiers so outcomes can be matched to predictions when they arrive.

Label sampling

Have experts label a small random sample quickly to estimate current accuracy.

Service level objectives

A service level objective is a measurable target for reliability that the team commits to, such as 99.9 percent of requests answered under 200 milliseconds.

Indicators

Choose the signals users feel: availability, latency and freshness of predictions.

Error budget

The allowed amount of failure. Spending it fast pauses risky releases.

Model objectives

Add targets for quality, such as weekly precision staying above an agreed level.

An error budget, worked out and burned down

OBJECTIVE

99.9%

of requests succeed over 30 days

BUDGET IN TIME

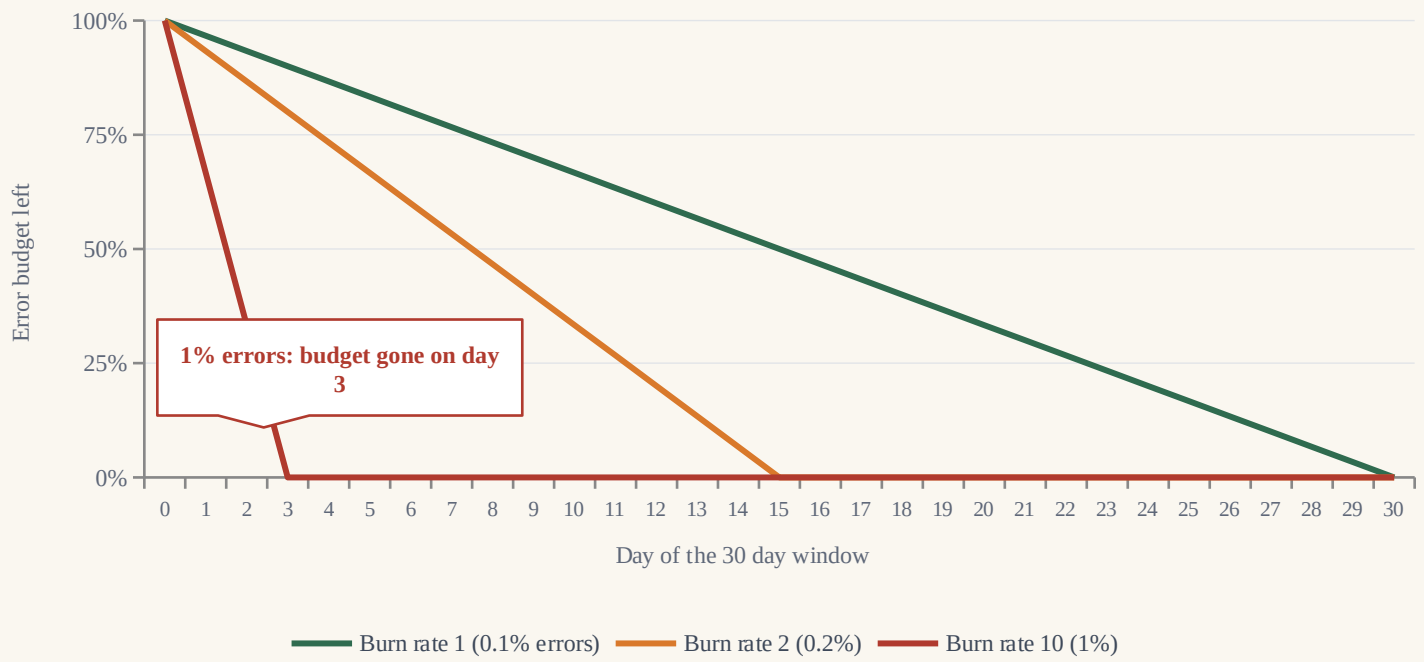
43.2 min

30 × 24 × 60 × 0.001

BUDGET IN REQUESTS

10,000

failures allowed out of 10 million



More than 50% left

Ship features and experiments freely.

Below 25% left

Only fixes and low-risk changes.

Exhausted

Freeze risky releases; reliability work first.

SECTION 13 · MONITORING

Alerting without alarm fatigue

1 **Alert on symptoms**

Page people for user impact such as high errors, not every internal blip.

2 **Severity levels**

Distinguish urgent pages from tickets that can wait until morning.

3 **Runbooks**

Every alert links to clear steps for diagnosis and response.

4 **Owners**

Route each alert to a named team with on call coverage.

5 **Tune thresholds**

Review noisy alerts regularly and remove or adjust them.

6 **Test alerts**

Trigger alerts deliberately to prove they reach the right people.

Monitoring tools landscape

Category	Examples
Metrics and dashboards	Prometheus, Grafana, cloud native monitoring services
Logs and traces	OpenTelemetry, Elasticsearch, Loki, Jaeger, Datadog
ML monitoring	Evidently, NannyML, Arize, Fiddler, cloud model monitors
Data quality	Great Expectations, Soda, Pandera, dbt tests
Alerting	Alertmanager, PagerDuty, Opsgenie, Slack integrations

Tools change often. Prioritise open standards so you can switch vendors without rewriting instrumentation.

A useful model dashboard

Health summary	Availability, error rate and p95 latency against objectives.
Traffic	Requests per minute by client, version and region.
Inputs	Drift scores and missing value rates for the top features.
Outputs	Prediction distribution and average confidence over time.
Quality	Accuracy or precision as labels arrive, with slice breakdowns.
Business	The outcome metric the model was built to improve.

Design tip

Put the answer to is it working at the top. Details live lower down for investigation. Always show the model version beside every chart.

SECTION 13 · MONITORING

Monitoring pitfalls

PITFALL	BETTER PRACTICE
⚠ Monitoring only CPU and uptime.	✓ Add data, prediction and business layers.
⚠ Alerting on every statistically significant drift.	✓ Combine drift with impact thresholds and review weekly.
⚠ Logging raw personal data for debugging.	✓ Mask sensitive fields and enforce retention rules.
⚠ Dashboards nobody owns or reads.	✓ Assign owners and review dashboards in regular meetings.

Section recap

KEY TAKEAWAYS

- Models can fail silently while services look healthy
- Monitor infrastructure, service, data, model and business layers
- Data drift warns early, concept drift needs labels
- Plan for delayed ground truth with proxies and joins
- Use objectives and error budgets to guide releases
- Alert on symptoms with runbooks and owners

TRY THIS

Sketch a monitoring dashboard for your model with one chart per layer and define two alerts with thresholds and owners.

NEXT UP

Section 14: retraining and managing the model lifecycle.

SECTION 14

14

Retraining and the model lifecycle

Keeping models useful as the world changes, and retiring them gracefully when it is time.

IN THIS SECTION

- Retraining triggers
- Continuous training pipelines
- Feedback loops
- Champion and challenger
- Data and model lineage
- Deprecation and retirement

SECTION 14 · LIFECYCLE

When should a model be retrained?

1 On a schedule

Weekly or monthly retraining suits steadily changing domains.

2 On drift

Retrain when monitored drift passes an agreed threshold.

3 On performance drop

Retrain when measured accuracy falls below target.

4 On new data

Retrain when enough fresh labelled data has accumulated.

5 On business change

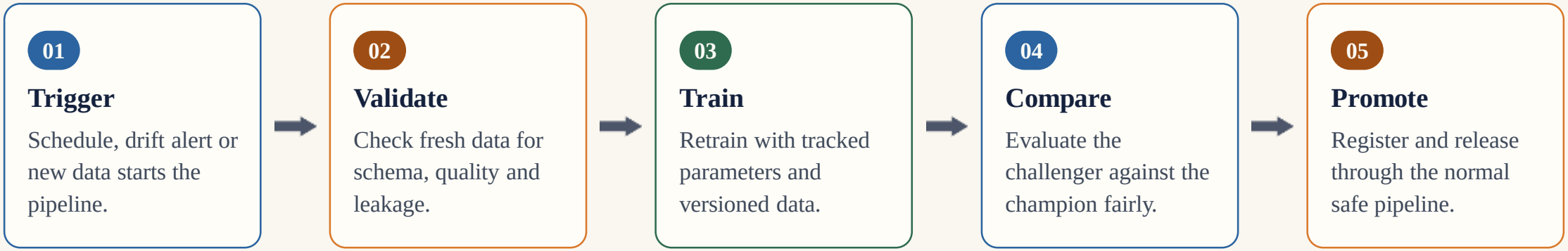
New products, regions or policies often demand retraining.

6 On request

Allow controlled manual triggers for investigations and fixes.

SECTION 14 · LIFECYCLE

A continuous training pipeline



Continuous training should never bypass the release gates. A retrained model is still a new model that can fail in new ways.

SECTION 14 · LIFECYCLE

Champion and challenger

DEFINITION

The champion is the model currently serving production. A challenger is a candidate that must prove itself better, on agreed criteria, before replacing the champion.

EXAMPLE

Each month a retrained churn model challenges the champion on the same holdout set and in a shadow test. It is promoted only if precision improves and no customer segment gets worse.

REMEMBER

A newer model is not automatically a better model.

SECTION 14 · LIFECYCLE

Feedback loops and their dangers

When predictions influence the data later used for training, the model can reinforce its own mistakes or biases over time.

Example

A model approves only some applicants, so outcomes are learned only for those approved.

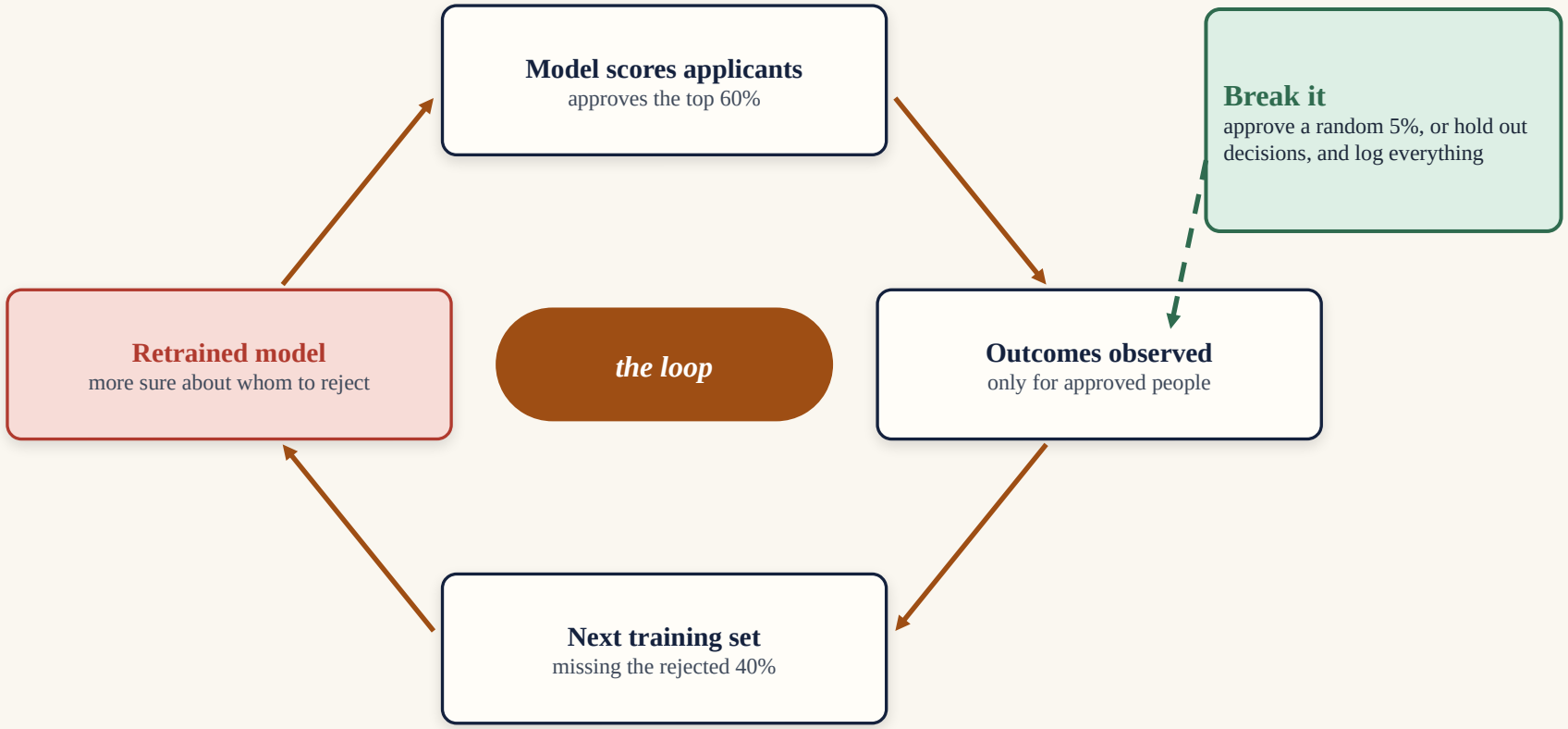
Consequences

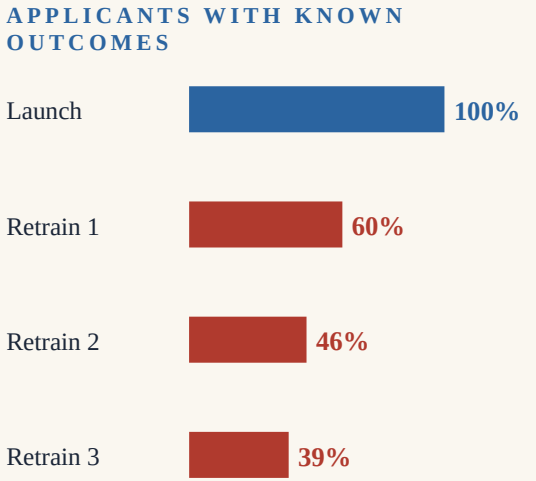
Blind spots grow, fairness can degrade and performance estimates become optimistic.

Mitigations

Hold out random exploration traffic, log decisions and use careful evaluation methods.

How a lending model teaches itself a blind spot





Illustrative. Offline metrics look better each round while the blind spot grows.

! Any model whose decisions shape its own future training data needs exploration traffic and decision logs from day one.

SECTION 14 · LIFECYCLE

Lineage: knowing where every model came from

Data version	Snapshot or query plus hash of the training data.
Code version	Git commit of training and feature code.
Parameters	Hyperparameters, seeds and configuration used.
Artifact	Model file, checksum and registry version.
Deployment	Which environments served it, when, and with what traffic share.

Why it matters

When a regulator, customer or engineer asks why a prediction was made, lineage lets you reconstruct the exact model, data and code involved.

SECTION 14 · LIFECYCLE

Retiring a model gracefully

1. **Decide.** Agree retirement because of replacement, low value or new regulation.
2. **Announce.** Tell consumers the date and the replacement or alternative.
3. **Monitor callers.** Log remaining traffic and contact clients still using the old version.
4. **Switch off.** Remove traffic, then decommission infrastructure and alerts.
5. **Archive.** Keep artifacts, model card and lineage for audits and comparisons.
6. **Clean up.** Delete unneeded data in line with retention and privacy rules.

SECTION 14 · LIFECYCLE

Lifecycle pitfalls

PITFALL	BETTER PRACTICE
⚠ Retraining automatically and deploying without review.	✓ Route every retrained model through the same gates and canary.
⚠ Retraining on data shaped by the model itself.	✓ Keep exploration data and watch for feedback loops.
⚠ Losing track of which data trained which model.	✓ Record data versions and lineage in the registry.
⚠ Zombie models that run forever unnoticed.	✓ Review model inventory and retire those with no owner or value.

Section recap

KEY TAKEAWAYS

- Retrain on schedules, drift, performance or new data
- Continuous training still passes every release gate
- Challengers must beat champions on agreed criteria
- Watch for feedback loops created by predictions
- Lineage makes every prediction explainable after the fact
- Retire models deliberately and archive their history

TRY THIS

Write a one page retraining policy for your model: triggers, comparison rules, approvers and retirement criteria.

NEXT UP

Section 15: deploying models to edge and mobile devices.

SECTION 15

15

Edge and mobile deployment

Running models where the data is born: phones, browsers, cameras, vehicles and machines.

IN THIS SECTION

- Why deploy at the edge
- Device constraints
- Mobile runtimes
- Browser inference
- Updating models over the air
- Privacy at the edge

Why deploy at the edge?

1 Latency
No network round trip, so responses feel instant.

2 Offline use
Works in tunnels, fields, aircraft and remote sites.

3 Privacy
Sensitive data such as faces or voices can stay on the device.

4 Bandwidth
Send only results or rare events instead of raw video.

5 Cost
Inference runs on hardware users already own.

6 Resilience
Keeps working when cloud services are unavailable.

SECTION 15 · EDGE AND MOBILE

Constraints you must design for

Edge devices are small, varied and battery powered. A model that runs comfortably on a server may be far too large or slow on a phone.

Memory and storage

Models often need to be a few megabytes to tens of megabytes.

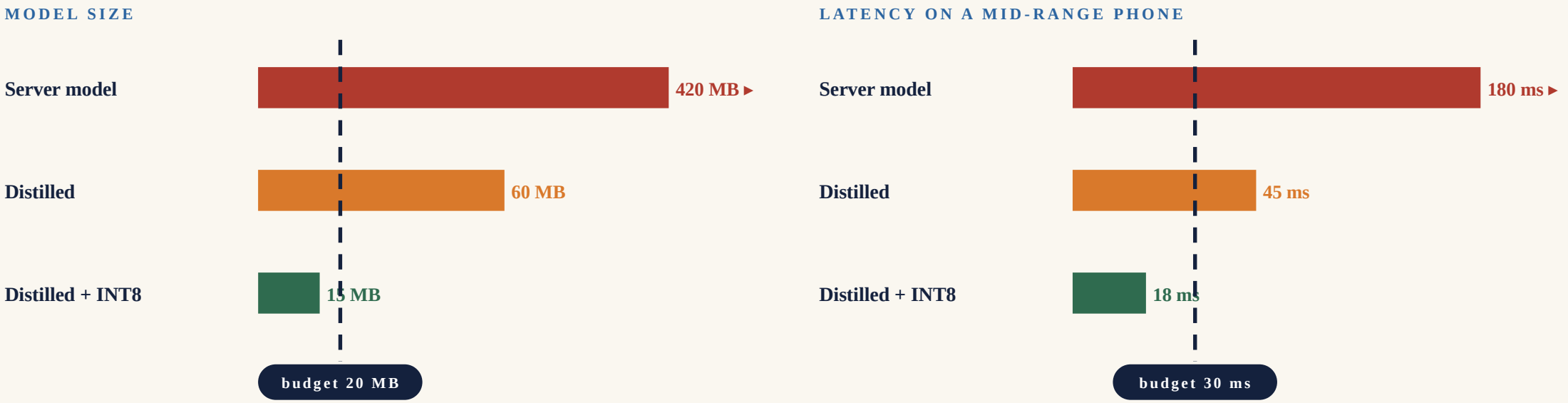
Compute and power

Heavy inference drains batteries and heats devices, causing throttling.

Fragmentation

Different chips, operating systems and accelerators behave differently.

Shrinking a server model to fit a phone's budget



Bars capped at 100; ▸ means off the scale. Illustrative figures.

ACCURACY ON THE SAME TEST SET

92.4% **Server model**
FP32 · original · 21× too big

91.8% **Distilled**
smaller student model · still 3× too big

91.5% **Distilled + INT8**
quantised for the NPU · meets every budget

On device runtimes

Runtime	Platforms	Notes
LiteRT (TensorFlow Lite)	Android, iOS, embedded Linux, microcontrollers	Google renamed TensorFlow Lite as LiteRT
Core ML	Apple devices	Uses the Apple Neural Engine where available
ONNX Runtime Mobile	Android, iOS	Shares the ONNX model format with servers
ExecuTorch	Mobile and edge	PyTorch solution for on device inference
OpenVINO	Intel edge hardware	Optimised for Intel CPUs, GPUs and NPUs

Choose the runtime that matches your device fleet and training framework, then test on the slowest device you support.

Inference in the browser

Libraries such as TensorFlow.js, ONNX Runtime Web and Transformers.js run models directly in the browser using WebAssembly, WebGL or WebGPU.

Benefits

No server cost per prediction and user data never leaves the page.

Limits

Download size matters, so models must be small and cached sensibly.

Good fits

Image filters, text classification, autocomplete and accessibility features.

Over the air model updates

01 — Publish
Upload a signed model file with version and compatibility metadata.

02 — Check
Devices ask for updates on a schedule or at app start.

03 — Download
Fetch in the background on suitable network and battery.

04 — Verify
Check signature and checksum before activation.

05 — Activate
Swap atomically and keep the old model for rollback.

Roll updates out to a small share of devices first and collect anonymous health metrics before a full release.

Privacy and federated learning

Edge deployment keeps raw data local. Federated learning goes further by training across many devices and sharing only model updates.

On device personalisation

Adapt a model to one user locally without uploading their data.

Federated learning

Devices compute updates, a server aggregates them into a better global model.

Extra protection

Secure aggregation and differential privacy reduce what updates can reveal.

SECTION 15 · EDGE AND MOBILE

Edge pitfalls

PITFALL	BETTER PRACTICE
⚠ Testing only on the latest flagship phone.	✓ Benchmark on low end and older devices you support.
⚠ Shipping models inside app releases only.	✓ Add over the air updates with version checks.
⚠ Unprotected model files inside apps.	✓ Assume extraction is possible and avoid embedding secrets.
⚠ No telemetry from devices.	✓ Collect privacy respecting metrics on latency and failures.

Section recap

KEY TAKEAWAYS

- Edge gives speed, offline use, privacy and lower cost
- Design for memory, power and device variety
- Pick runtimes that match devices and frameworks
- Browsers can run small models with no server
- Update models over the air with signing and rollback
- Federated learning trains without centralising raw data

TRY THIS

Convert a small image model to an on device format and measure size and latency on a phone or in a browser.

NEXT UP

Section 16: deploying large language models.

SECTION 16

16

Deploying large language models

What changes when the model has billions of parameters and answers in free text.

IN THIS SECTION

- How LLM serving differs
- APIs versus self hosting
- Serving engines
- Memory and batching
- RAG in production
- Evaluation, guardrails and cost

SECTION 16 · LLMS

Classic ML versus LLM serving

CLASSIC ML

Small, fast, structured

- Megabytes to a few gigabytes
- Single fast forward pass per request
- Structured inputs and outputs
- Accuracy measured against labels

LLMS

Huge, generative, open ended

- Gigabytes to hundreds of gigabytes
- Generates output token by token
- Free text in and free text out
- Quality needs nuanced evaluation

VERDICT LLM deployment shares the same foundations, but memory, generation latency, evaluation and safety all need extra attention.

Hosted API versus self hosted LLM

HOSTED API

Call a provider

- Fastest path to production
- Access to frontier capability
- Pay per token, little operations work
- Data handling depends on provider terms

SELF HOSTED

Run open weight models

- Full control of data and versions
- Can be cheaper at high steady volume
- Needs GPUs and serving expertise
- You own scaling, updates and safety

VERDICT Many teams start with hosted APIs, then self host specific workloads when volume, privacy or customisation justify the effort.

LLM serving engines

Engine	Known for	Consider
vLLM	Efficient memory paging, high throughput	Popular open source default
Hugging Face TGI	Production server for open models	Check current maintenance status
TensorRT LLM	Peak performance on NVIDIA GPUs	More build and tuning effort
SGLang	Fast structured generation and caching	Rapidly evolving project
llama.cpp and Ollama	CPU and local machine inference	Best for small or local use

This space moves monthly. Benchmark engines again on your own model and prompts before each major decision.

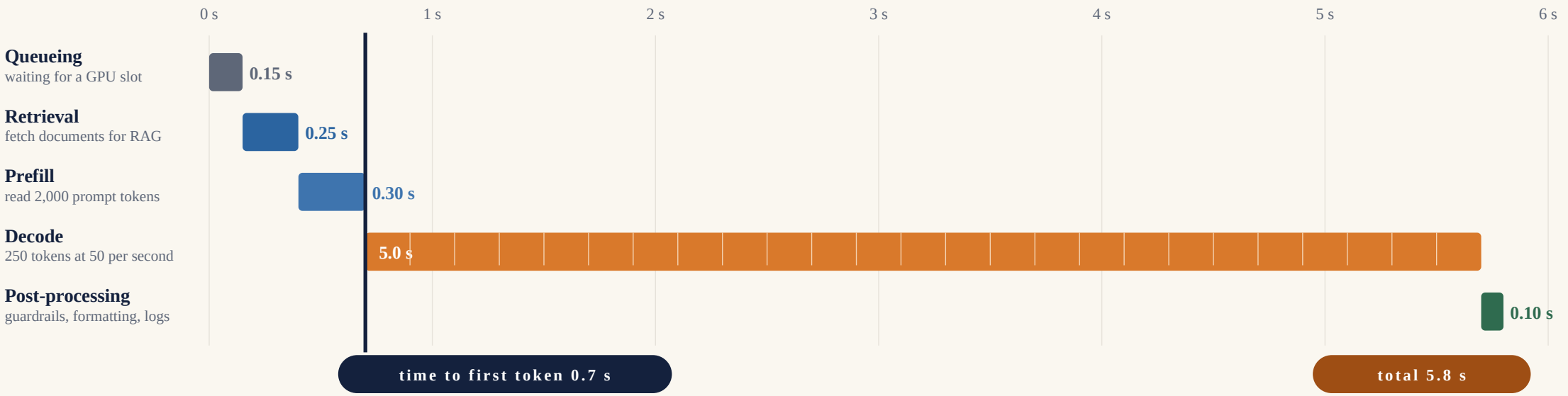
Where LLM latency comes from

Queueing	Waiting for a free slot on a busy GPU.
Prefill	Reading the whole prompt. Long prompts cost more.
Decode	Generating each new token one after another.
Retrieval	Fetching documents for RAG before generation starts.
Post processing	Guardrails, formatting and logging after generation.

Two numbers

Track time to first token, which users feel as responsiveness, and tokens per second, which decides how long full answers take. Stream tokens to the user.

Anatomy of one 5.8 second LLM answer



THE LEVER FOR EACH STAGE

Queueing	Retrieval	Prefill	Decode	Post
continuous batching, more replicas	cache embeddings, faster index	shorter prompts, prefix caching	cap output length, smaller model, speculative decoding	log asynchronously

SECTION 16 · LLMS

Memory and batching techniques

1 KV cache

Stores attention state so earlier tokens are not recomputed for each new token.

2 Paged attention

Manages cache memory in pages to waste less GPU memory.

3 Continuous batching

Adds and removes requests from a running batch at every step.

4 Quantised weights

Four or eight bit weights fit bigger models on fewer GPUs.

5 Prefix caching

Reuses computation for shared prompt beginnings such as system prompts.

6 Speculative decoding

A small draft model proposes tokens that the large model verifies.

Why the KV cache, not the weights, limits how many users fit

BYTES PER TOKEN = 2 (K and V) × layers × KV heads × head size × bytes per value

Full multi-head attention

older 7B design · 32 KV heads

`2 × 32 × 32 × 128 × 2`

0.5 MiB	2 GiB	≈ 4
per token	per 4,096-token chat	chats that fit

Grouped-query attention

8B design · 8 KV heads

`2 × 32 × 8 × 128 × 2`

128 KiB	512 MiB	≈ 18
per token	per 4,096-token chat	chats that fit

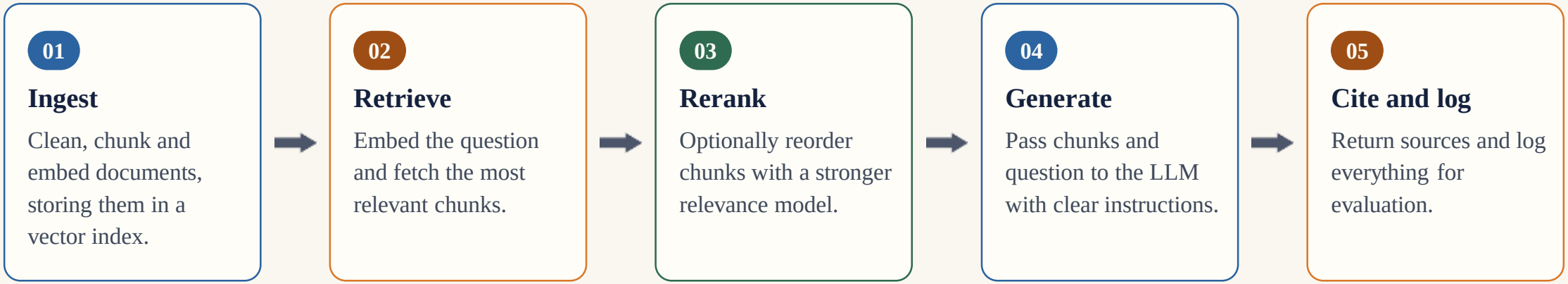
ONE 24 GB GPU



Paged attention stops each chat reserving its maximum length up front, so real capacity is often higher. Ignores activations and overhead.

SECTION 16 · LLMS

Retrieval augmented generation in production



Most RAG failures are retrieval failures. Monitor retrieval quality separately from answer quality.

Evaluating LLM applications

There is rarely one correct answer, so LLM evaluation combines automated checks, model based judging and human review on curated test sets.

Golden datasets

Curated questions with reference answers or rubrics, versioned like code.

Automated scoring

Exact checks, similarity measures and LLM as a judge with validated rubrics.

Human review

Expert sampling of real traffic catches failures that metrics miss.

Guardrails for LLM systems

1 Input filtering
Detect abuse, personal data and prompt injection attempts.

2 Output filtering
Check for unsafe content, data leakage and policy breaches.

3 Grounding checks
Verify answers are supported by retrieved sources.

4 Structured output
Validate JSON or schema outputs before passing them to other systems.

5 Least privilege tools
Agents and tools get only the permissions they truly need.

6 Human in the loop
Require approval for high impact actions such as payments.

Managing LLM cost

LLM costs scale with tokens processed, so prompt design, routing and caching often matter more than hardware discounts.

Shorter prompts

Trim instructions and retrieved context to what actually improves answers.

Model routing

Send simple requests to small models and hard ones to large models.

Caching

Cache answers to repeated questions and reuse shared prompt prefixes.

SECTION 16 · LLMS

LLMOps essentials

- Version prompts like code, with reviews
- Pin model versions, not moving aliases
- Keep a golden evaluation set per use case
- Run evaluations on every prompt or model change
- Track time to first token and tokens per second
- Monitor cost per request and per user
- Log prompts and responses with privacy controls
- Test for prompt injection regularly
- Provide user feedback buttons and review them
- Plan fallbacks when a provider is unavailable

LLM deployment pitfalls

PITFALL	BETTER PRACTICE
⚠ Judging quality from a handful of demo prompts.	✓ Evaluate on a broad, versioned test set before release.
⚠ Letting model output trigger actions directly.	✓ Validate outputs and require approval for risky actions.
⚠ Unbounded context windows that explode cost.	✓ Budget tokens per request and monitor spend daily.
⚠ Upgrading the underlying model silently.	✓ Pin versions and rerun evaluations before switching.

Section recap

KEY TAKEAWAYS

- LLMs share deployment foundations but stretch memory, latency and evaluation
- Choose hosted APIs or self hosting based on volume, privacy and control
- Serving engines use caching and continuous batching for efficiency
- RAG quality depends on retrieval quality
- Guardrails and evaluation are part of the product
- Manage cost through prompts, routing and caching

TRY THIS

Build a tiny RAG service over ten documents and create a 20 question golden set to evaluate it after every change.

NEXT UP

Section 17: security, privacy and governance.

SECTION 17

17

Security, privacy and governance

Protecting models, data and people, and proving it to customers and regulators.

IN THIS SECTION

- Threats unique to ML
- Securing the pipeline
- Access and secrets
- Privacy and data protection
- Regulation and the EU AI Act
- Fairness, explainability, audit

Threats unique to ML systems

1 Adversarial inputs
Carefully crafted inputs that push a model into wrong predictions.

2 Data poisoning
Malicious records inserted into training data to corrupt behaviour.

3 Model extraction
Repeated queries used to copy a model or steal its behaviour.

4 Membership inference
Attacks that reveal whether a person was in the training data.

5 Prompt injection
Hidden instructions that hijack LLM applications and agents.

6 Supply chain
Compromised libraries, base images or downloaded model files.

Securing the ML supply chain

Data sources	Verify provenance, access controls and integrity checks.
Dependencies	Pin versions, scan for vulnerabilities and use trusted mirrors.
Pretrained models	Prefer safe formats, verify checksums and review licences.
Build system	Isolated, reproducible builds with signed outputs.
Registry	Signed images and models, verified again at deploy time.

Trust nothing by default

Treat every external dependency, dataset and pretrained model as untrusted until verified. Frameworks such as SLSA describe levels of build integrity.

Access control and secrets

- Authenticate every caller of the model API
- Use short lived tokens over static keys
- Grant least privilege to services and people
- Store secrets in a dedicated secret manager
- Rotate credentials automatically
- Encrypt data in transit and at rest
- Rate limit to slow extraction attempts
- Log access and review it regularly
- Separate production from development access
- Remove access promptly when people leave

Privacy and data protection

In the UK and EU, the GDPR and UK GDPR govern personal data used to train and run models, including rights around automated decisions.

Data minimisation

Use only the personal data you genuinely need, and pseudonymise where possible.

Lawful basis

Document why you may process the data and tell people clearly.

Automated decisions

Significant solely automated decisions carry extra rights, including human review.

The EU AI Act in brief

The EU AI Act entered into force in 2024 and applies obligations in phases over several years, based on the risk level of each AI system.

Risk tiers

Some practices are prohibited, high risk systems face strict duties, and others carry transparency rules.

High risk duties

Risk management, data governance, documentation, logging, human oversight and accuracy.

Check dates

Timelines and guidance continue to evolve, so confirm current obligations with legal experts.

The EU AI Act's four risk tiers and its key dates



Classify every model in your inventory against these tiers.

KEY DATES

1 Aug 2024
Act enters into force

2 Feb 2025
Prohibitions and AI literacy apply

2 Aug 2025
General-purpose AI model duties

2 Aug 2026
Transparency duties apply

2 Dec 2027
Stand-alone high-risk systems*

2 Aug 2028
High-risk AI in regulated products*

** As moved by the 2026 AI Omnibus amendment. Check current guidance with legal experts.*

Fairness in production

A model can be accurate on average and still treat some groups worse. Fairness must be measured before release and monitored after it.

Measure slices

Compare error rates and outcomes across relevant groups and regions.

Choose definitions

Different fairness definitions conflict, so agree which matters for your use case.

Monitor over time

Group performance can drift even when overall metrics look stable.

Global versus local explanations

GLOBAL

How the model behaves overall

- Which features matter most in general
- Useful for validation and governance
- Supports model cards and reviews
- Example: overall feature importance

LOCAL

Why one prediction was made

- Which features drove this specific result
- Useful for customers and support teams
- Supports appeals and adverse action notices
- Example: SHAP values for one applicant

VERDICT Production systems often need both. Compute local explanations on demand or store them with predictions for later review.

Audit readiness

1 Model inventory
A list of every model in production with owner and purpose.

2 Decision logs
Inputs, outputs, version and explanations for important decisions.

3 Approvals
Who approved each release, when and based on what evidence.

4 Documentation
Model cards, data sheets and risk assessments kept current.

5 Incident records
What went wrong, impact and corrective actions taken.

6 Retention
Logs kept long enough for audits but no longer than allowed.

Security and governance pitfalls

PITFALL	BETTER PRACTICE
⚠ Loading pretrained models from unknown sources.	✓ Use trusted sources, safe formats and checksum verification.
⚠ Open prediction endpoints without authentication.	✓ Authenticate, authorise and rate limit every endpoint.
⚠ Treating compliance as a final sign off.	✓ Involve privacy and risk teams from project framing.
⚠ No record of why a decision was made.	✓ Log versions, inputs and explanations for key decisions.

Section recap

KEY TAKEAWAYS

- ML adds threats such as poisoning, extraction and prompt injection
- Secure data, dependencies, models, builds and registries
- Apply least privilege and managed secrets everywhere
- Respect GDPR rights including those on automated decisions
- Know where your systems sit under the EU AI Act
- Measure fairness, provide explanations and stay audit ready

TRY THIS

Run a threat modelling session for your model service. List assets, attackers, attack paths and one mitigation for each.

NEXT UP

Section 18: reliability and cost management.

SECTION 18

18

Reliability and cost

Designing model services that keep working when things break, at a price the business can sustain.

IN THIS SECTION

- Failure is normal
- Timeouts, retries and circuit breakers
- Graceful degradation
- Redundancy and recovery
- Incident response
- FinOps for ML

> *Everything fails eventually. Reliable systems are designed to fail gracefully and recover quickly.*

☆ Plan for slow dependencies, lost nodes, bad deployments and corrupted data, not only for the happy path.

Resilience patterns

1 Timeouts

Never wait forever on a dependency. Set budgets for each call.

2 Retries with backoff

Retry transient failures with growing delays and random jitter.

3 Circuit breakers

Stop calling a failing dependency for a while to let it recover.

4 Bulkheads

Isolate resources so one overloaded model cannot starve others.

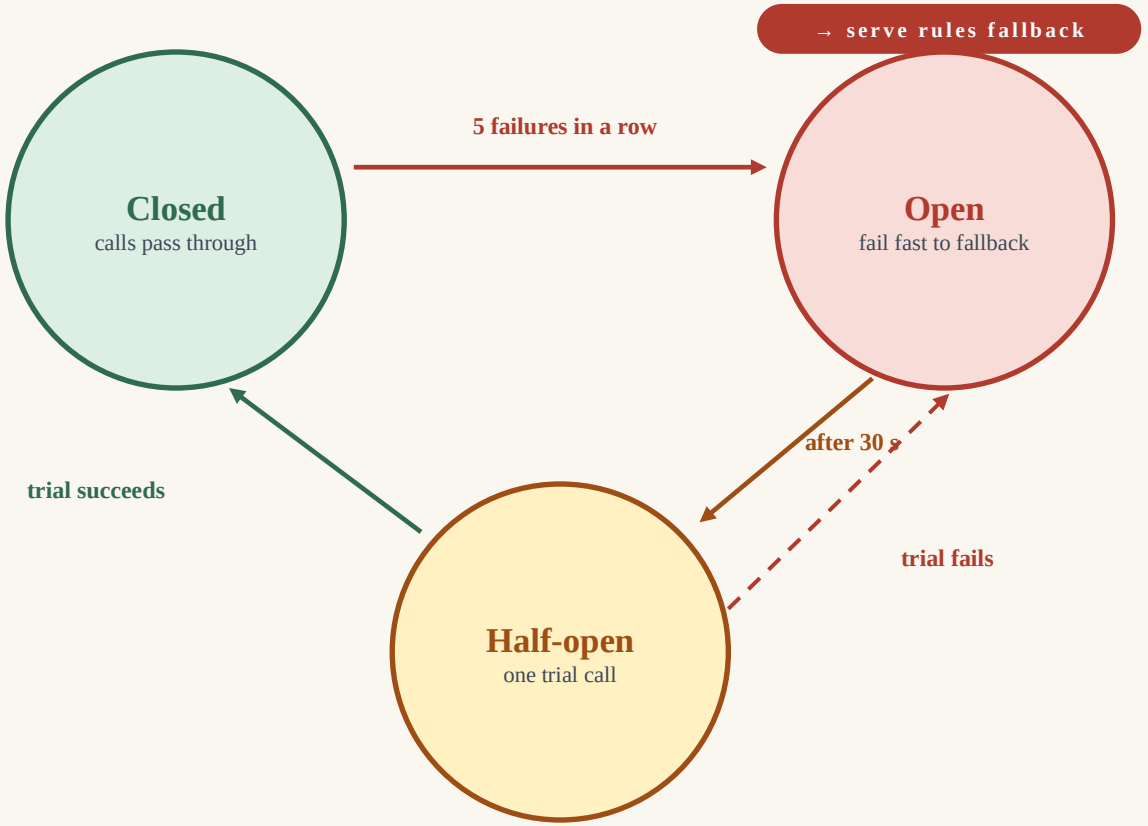
5 Load shedding

Reject low priority requests early when the system is saturated.

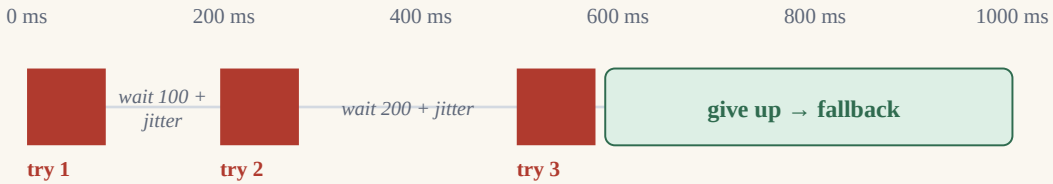
6 Idempotency

Make repeated requests safe so retries cannot cause duplicate effects.

Retries with backoff and a circuit breaker, working together



ONE CALL, THREE ATTEMPTS



Timeout 80 ms per attempt: never wait forever.

Backoff 100 → 200 → 400 ms between attempts.

Jitter random extra delay so clients do not retry in lockstep.

Idempotent a request ID makes repeats safe.

! Retries handle brief blips; the breaker handles real outages. Without it, retries multiply load on a service that is already down.

Graceful degradation with fallbacks

01

Primary

The full model returns a fresh, personalised prediction.

02

Cached

If it is slow, serve a recent cached prediction for that user.

03

Simple model

If no cache, use a lightweight backup model or rules.

04

Safe default

If all else fails, return a sensible default such as popular items.

05

Signal it

Mark degraded responses and alert when fallbacks are frequent.

Users rarely notice a slightly less personalised result. They always notice an error page.

Redundancy and disaster recovery

Multiple replicas	Several Pods spread across nodes for every service.
Multiple zones	Replicas in separate availability zones within a region.
Multiple regions	Standby or active capacity in another region for major outages.
Backups	Registry, feature store and configuration backed up and restorable.
Drills	Regular recovery exercises prove plans work in practice.

Two key targets

Recovery time objective is how quickly you must be back. Recovery point objective is how much recent data you can afford to lose. Both drive design and cost.

SECTION 18 · RELIABILITY AND COST

Incident response for model services

1. **Detect.** Alerts or users report a problem with errors, latency or prediction quality.
2. **Triage.** Assess impact and severity, and name an incident lead.
3. **Mitigate.** Roll back, switch to fallback or disable the model with a flag.
4. **Communicate.** Update stakeholders and affected teams at regular intervals.
5. **Resolve.** Fix the root cause and verify recovery with monitoring.
6. **Learn.** Hold a blameless review and track follow up actions to completion.

SECTION 18 · RELIABILITY AND COST

Chaos and resilience testing

Chaos engineering deliberately injects failures in a controlled way to check that systems behave as designed before a real incident does it for you.

Start small

Kill one replica in staging and confirm traffic continues smoothly.

Expand carefully

Add latency to the feature store, block a dependency or drain a node.

ML specific

Inject corrupted inputs or missing features and check validation and fallbacks.

SECTION 18 · RELIABILITY AND COST

FinOps for machine learning

1 Visibility
Tag resources and report cost per model, team and prediction.

2 Unit economics
Track cost per thousand predictions alongside value created.

3 Utilisation
Find idle GPUs and oversized instances every week.

4 Right model size
Use the smallest model that meets quality targets.

5 Pricing models
Mix on demand, reserved and spot capacity sensibly.

6 Accountability
Give each model owner a budget and a cost dashboard.

Reliability and cost pitfalls

PITFALL	BETTER PRACTICE
⚠ Retrying instantly and endlessly on failure.	✓ Use limited retries with exponential backoff and jitter.
⚠ A single replica for a critical model.	✓ Run at least two replicas across zones.
⚠ No fallback when the model is unavailable.	✓ Design cached, simple or default responses.
⚠ Discovering cost overruns at month end.	✓ Set budgets and daily alerts on spend per model.

Section recap

KEY TAKEAWAYS

- Design for failure with timeouts, retries and circuit breakers
- Degrade gracefully through layered fallbacks
- Spread replicas across zones and rehearse recovery
- Respond to incidents with clear roles and blameless reviews
- Test resilience deliberately with chaos experiments
- Manage cost per prediction as a first class metric

TRY THIS

Define a four level fallback chain for your model and calculate its monthly cost at today’s traffic.

NEXT UP

Section 19: an end to end case study.

SECTION 19

19

End to end case study

Deploying a card payment fraud model from first conversation to steady operation.

IN THIS SECTION

- The business problem
- Requirements and constraints
- Architecture
- Build and release
- Monitoring and incidents
- Lessons learned

SECTION 19 · CASE STUDY

The scenario



A fictional online retailer loses money to card fraud and wants to block risky payments at checkout without annoying genuine customers.

Current state

A rules engine flags payments, missing new fraud patterns and blocking many good customers.

Goal

Reduce fraud losses while keeping false declines low and checkout fast.

Model

A gradient boosted tree classifier trained on historical labelled transactions.

SECTION 19 · CASE STUDY

Requirements captured at framing

Requirement	Decision for this example
Pattern	Online, synchronous call during checkout
Latency budget	p99 under 100 milliseconds for the whole fraud check
Availability	99.95 percent, with a rules fallback if the model is down
Explainability	Top reasons stored for every declined payment
Labels	Chargebacks arrive weeks later, so proxies are needed
Privacy	Card numbers tokenised, no raw personal data in logs

Writing requirements as a table forces agreement before any code is written, and becomes the basis for tests and alerts.

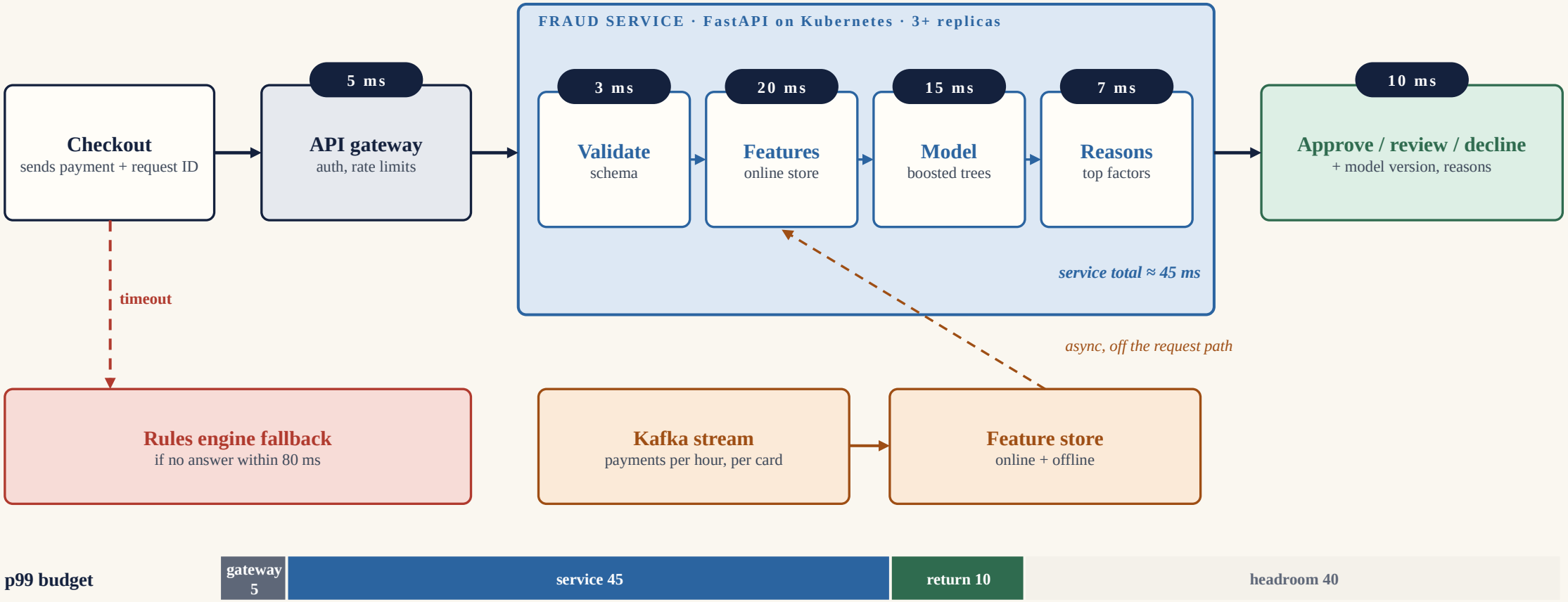
Target architecture

Checkout service	Calls the fraud API with payment details and a request identifier.
API gateway	Authentication, rate limiting and routing to the fraud service.
Fraud service	FastAPI on Kubernetes, model loaded at startup, three or more replicas.
Feature store	Online store for fresh features, offline store for training.
Stream pipeline	Kafka events update velocity features such as payments per hour.
Monitoring	Prometheus, Grafana, prediction logs and drift reports.

Design choices

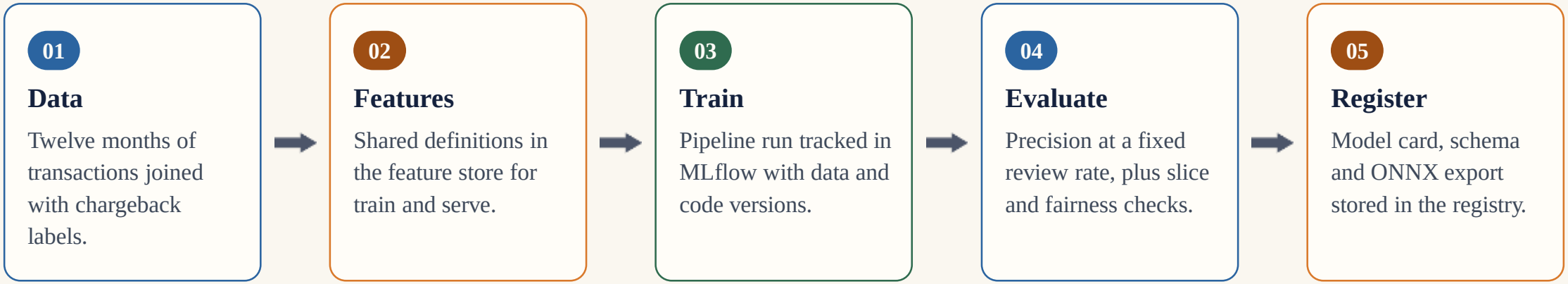
Features with long histories are precomputed in batch, while fresh velocity features update from the stream. The model service reads both in a single fast lookup.

The fraud check's architecture, with its 100 ms budget split



SECTION 19 · CASE STUDY

Building the model artifact



SECTION 19 · CASE STUDY

Fraud scoring endpoint with fallback

fraud_api.py

```
@app.post("/v1/score")
async def score(p: Payment):
    try:
        feats = await feature_store.get(p.card_token, timeout=0.03)
        risk = model.predict(p, feats)
        source = "model"
    except (TimeoutError, ModelError):
        risk = rules_engine.score(p)
        source = "fallback"
    log_prediction(p.request_id, risk, source, MODEL_VERSION)
    return {"risk": risk, "source": source,
            "model_version": MODEL_VERSION}
```

Time budget

Feature lookup gets 30 milliseconds, protecting the overall latency target.

Fallback

If features or model fail, the trusted rules engine still protects checkout.

Traceability

Every response records its source and model version for audits.

SECTION 19 · CASE STUDY

Release plan

1. **Shadow for two weeks.** Score all payments silently and compare against rules and later chargebacks.
2. **Review results.** Fraud analysts confirm catches and false alarms on a sample.
3. **Canary at 5 percent.** Model decisions apply to a small share of traffic with tight alerts.
4. **Expand gradually.** Move to 25, 50 and 100 percent over several days.
5. **Keep rules warm.** The rules engine remains ready as an instant fallback.
6. **Sign off.** Product, risk and engineering approve full rollout.

SECTION 19 · CASE STUDY

What the team monitors

1 Service health

Error rate, p99 latency and fallback rate against objectives.

2 Inputs

Drift on amount, country, device and velocity features.

3 Outputs

Decline rate and risk score distribution per hour.

4 Early proxies

Customer complaints and manual review outcomes.

5 Late truth

Chargebacks joined to predictions weekly for precision and recall.

6 Fairness

Decline rates across regions and customer segments.

SECTION 19 · CASE STUDY

An incident, and what it taught

⚠ Months later, a payment provider changed how it reported device type. Drift alerts fired on one feature and decline rates crept up in one region.

Mitigation

The team switched that region to the fallback using a feature flag within minutes.

Root cause

An upstream schema change was not caught because validation allowed new categories silently.

Fix

Stricter schema tests, a contract with the provider and a retrained model with the new values.

SECTION 19 · CASE STUDY

Lessons from the case study

The model was the easy part. Requirements, features, fallbacks and monitoring decided whether the project succeeded.

Design for failure

The rules fallback turned a potential outage into a minor, controlled dip.

Shadow first

Two weeks of silent scoring built trust with analysts before any customer impact.

Watch upstream

Many model incidents start as data changes in systems you do not own.

SECTION 19 · CASE STUDY

Section recap

KEY TAKEAWAYS

- Frame requirements explicitly before building
- Share feature definitions between training and serving
- Protect latency budgets with timeouts and fallbacks
- Release through shadow and canary stages
- Monitor proxies while waiting for delayed labels
- Treat upstream data changes as a top risk

TRY THIS

Adapt this case study to a model from your own work. Rewrite the requirements table and the release plan for it.

NEXT UP

Section 20: checklists, glossary and your next steps.

SECTION 20

20

Checklists, glossary and next steps

Practical tools to reuse on every deployment, and a path for continued learning.

IN THIS SECTION

- Pre deployment checklist
- Launch day checklist
- Post launch checklist
- Anti patterns
- Glossary
- Final knowledge check

SECTION 20 · TOOLKIT

Pre deployment checklist

- [ ] Business goal and success metric agreed
- [ ] Deployment pattern chosen and justified
- [ ] Model packaged with preprocessing
- [ ] Schema and input validation in place
- [ ] Model card and risk assessment complete
- [ ] Unit, data, behavioural and contract tests pass
- [ ] Load test meets latency budget
- [ ] Security scan shows no critical issues
- [ ] Monitoring dashboard and alerts ready
- [ ] Rollback and fallback plan documented

SECTION 20 · TOOLKIT

Launch day checklist

- [ ] Release window agreed with stakeholders
- [ ] Feature flags and kill switch confirmed
- [ ] On call engineer and incident lead named
- [ ] Communication channel open for updates
- [ ] Previous version ready for instant rollback
- [ ] Rollback triggers written and agreed
- [ ] Canary or shadow configuration verified
- [ ] Stakeholders know how to report issues
- [ ] Dashboards open and alerts tested
- [ ] Go or no go decision recorded

SECTION 20 · TOOLKIT

Post launch checklist

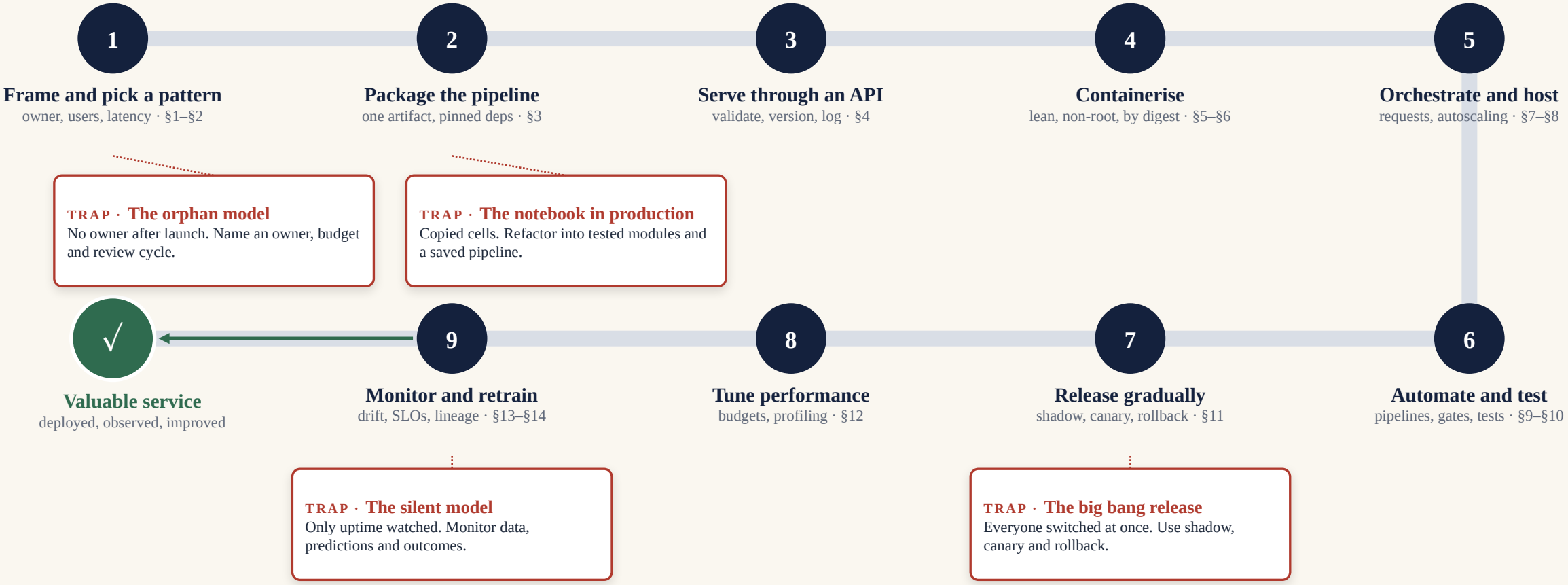
- [ ] Error rate and latency stable for a week
- [ ] Prediction distribution matches expectations
- [ ] No unexpected drift on key features
- [ ] Business metric trending as planned
- [ ] Fairness slices reviewed
- [ ] Cost per prediction within budget
- [ ] Alert noise reviewed and tuned
- [ ] Retraining schedule and triggers active
- [ ] Lessons captured in a short review
- [ ] Documentation updated with real behaviour

SECTION 20 · TOOLKIT

Deployment anti patterns

PITFALL	BETTER PRACTICE
The notebook in production: copying cells into a server script.	Refactor into tested modules and a saved pipeline.
The silent model: no monitoring beyond uptime.	Monitor data, predictions and business outcomes.
The big bang release: everyone switched at once.	Use shadow, canary and automatic rollback.
The orphan model: no owner after launch.	Name an owner, a budget and a review cycle for every model.

The whole deployment journey, with its four traps



SECTION 20 · TOOLKIT

Glossary, part one

Term	Meaning
Artifact	A saved, versioned output such as a model file or container image
Batch prediction	Scoring many records on a schedule and storing results
Canary	Releasing to a small share of traffic before everyone
Concept drift	Change in the relationship between inputs and the right answer
Container	A running, isolated instance of an image
Data drift	Change in the distribution of input features
Feature store	Shared system that serves features for training and serving
Inference	Using a trained model to make a prediction

Glossary, part two

Term	Meaning
Latency	Time taken to return one prediction
MLOps	Practices that make ML delivery reliable and repeatable
Model card	Document describing a model’s purpose, data, performance and limits
Model registry	Store of model versions, metadata and release stages
Quantisation	Using fewer bits for weights to shrink and speed up models
Shadow deployment	Running a new model on real traffic without using its output
SLO	Service level objective, a measurable reliability target
Throughput	Number of predictions completed per second

SECTION 20 · TOOLKIT

Final knowledge check 1

? Your new model passes offline tests but you worry about unknown real world behaviour. Which release strategy gives evidence with no user risk?

MODEL ANSWER

A shadow deployment. The new model receives a copy of real traffic and its predictions are logged and compared with the current model and later outcomes, but never returned to users. Once results look good, move to a canary release.

WHY IT MATTERS

It turns uncertainty into evidence before anyone is affected.

SECTION 20 · TOOLKIT

Final knowledge check 2

? Inputs to your model look exactly like training data, yet accuracy measured from fresh labels is falling. What is the likely cause?

MODEL ANSWER

Concept drift. The relationship between inputs and outcomes has changed even though input distributions have not. The fix is to gather recent labelled data, retrain, and compare the challenger against the champion before promoting it.

WHY IT MATTERS

It shows why data drift monitoring alone is not enough.

SECTION 20 · TOOLKIT

Final knowledge check 3

? A fraud API must answer within 100 milliseconds, but the feature store sometimes takes 300 milliseconds. What should the service do?

MODEL ANSWER

Set a strict timeout on the feature lookup and fall back gracefully, for example to cached features, a simpler model or a rules engine. Log and alert on the fallback rate, and work with the feature store owners to fix the slow path.

WHY IT MATTERS

Reliability comes from designing for slow dependencies, not hoping they never happen.

SECTION 20 · TOOLKIT

Your learning path from here

01

Practise

Deploy one small model end to end with the checklists.

02

Automate

Add CI, a registry and a canary release.

03

Observe

Instrument drift, quality and business dashboards.

04

Scale

Move to Kubernetes or a managed platform.

05

Specialise

Go deeper into LLMs, edge or governance.

Recommended reading: Designing Machine Learning Systems by Chip Huyen, Reliable Machine Learning from O’Reilly, and Google’s Rules of Machine Learning.

Great models create value only when they are deployed, observed and improved with care.

☆ Thank you for completing this tutorial. Use the checklists, revisit the sections as you build, and ship your next model with confidence.